

Renewable Energy Supply Steadiness

Project Work 1 Report

Project Supervisor:

Prof. Dr. Stefan Mátéfi-Tempfli

Authors

Han Lin Aung	22201576
Charles Albert Watiemeh Cobb	22307151
Ye Yint Myint Htwe	22209692
Kaung Htet San	22201794
Elsa Izeti	22211485
Toluwalade Lawal	22206436
Ahmed Fouad	22309260
Amos Addai Worae	22208595
Ahmed Saafan	22300578
Asrin Nakipoglu	22308182

Deggendorf Institute of Technology

European Campus Rottal-Inn

22.01.2025

Table of Contents

Abstract	1
Introduction	2
Solar Energy usage by a solar panel (Focus on diffused solar radiation)	3-12
Wind Team: Improvement of the Mathematical Wind Profile Model	13-25
Coding Team	26-36
References	37-39
Appendix	40-46

Abstract

This project is a continuation of the work done in previous semesters. Since one of this project's aims is to utilize the wind energy potential around Germany, wind power calculations play a crucial role in the accuracy of the project results. An important variable that directly affects said calculations is the speed of the wind entering the wind turbine. The wind part of this project focused primarily on the improvement of the mathematical model responsible for the estimation of this wind speed. Besides an improved model, different reasons that could decrease the accuracy of the estimation were investigated. As a result, a combination of two mathematical models that give more accurate estimations was achieved. The programming team implemented both models using Python, enabling us to calculate the wind power at several weather stations simultaneously, an additional advantage was the ability to combine the power generated at several station into one excel sheet, with the use of a python library called 'Pandas'.

Germany has made significant strides in transitioning to renewable energy over the past decade, with solar energy playing a crucial role. The cumulative installed capacity of solar energy moved from 38 GW in 2015 to 82.7 GW in 2024 and reaching 72.2 TWh solar energy production, which accounted for 33% of the electricity mix. The benefits of this transition are very well known, but chief amongst them is the reduction in fossil fuels and carbon emissions. With knowledge of this, one can fully attest to the fact that Germany is really pushing for a 100% green electricity supply. It is therefore necessary to estimate whether or not that will be possible with the available wind and solar resources. To do so, an assessment of both resources is done to understand the capacity/potential of the resources. For Solar, the Perez Diffuse Model is selected as the model to accurately determine the diffuse irradiance of the sun per location selected and the power thereof. The selection is made based on computational analysis and Pearson's correlation. The selected model, Perez Model, shows that the production capacity of Germany using some selected locations would suffice the consumption rate.

Introduction

Solar energy, a renewable energy source, plays an important part in today's energy market. This is because it is a "carbon-free" energy solution that produces no greenhouse gas emissions contributing to climate change after its initial setup [1]. Solar energy technology is classified into two key applications: solar thermal and solar PV, but almost all solar energy today comes from solar photovoltaics (PV). PV devices, often known as solar cells or panels, are electronic devices that convert sunlight into electrical power. PVs are also one of the fastest-developing renewable energy technologies today. It is expected to play a key role in the global electricity generation mix in the long term [2]. Solar energy can reach a solar panel through two main components of solar radiation: direct radiation and diffuse radiation. Solar panels utilise all these two components of solar radiation, but their efficiency is influenced by factors such as the angle of sunlight, cloud cover, and atmospheric conditions. For example, on a clear sunny day, direct radiation contributes the most, while on cloudy days, diffuse radiation becomes a critical source of energy for the panel. In our project, our solar group is focusing on the diffuse radiation component of global radiation. We are tasked with researching various diffuse radiation models to identify efficient ones and then creating a model based on these findings. For our analysis, we selected three models: the Perez Model, the Reindl Model, and the Hay and Davies Model. These models will be implemented in programming software such as MATLAB or Python to analyse their behaviour. We will then compare the results and determine the most suitable and efficient model among the three. Most of the data required to build the model equations is sourced from Deutscher Wetterdienst, while some parameters are calculated using online tools such as the NOAA Solar Geometry Calculator. After finalising the models, we estimated the values of diffuse irradiance on a tilted surface of the panel. This data is then handed over to the Programming Team, who will use it to calculate the consumption rate based on our solar diffuse model.

Wind energy, along with solar energy, is an essential renewable energy resource in Germany. In 2023, wind energy was the leading source of electricity production in Germany, surpassing sources such as brown coal and accounting for a net 32% portion of the produced energy (both onshore and offshore) [35]. This portion increased further to 33% in 2024, remaining a crucial energy resource [36]. If we only focus on the onshore wind turbines in Germany, the number of them installed by the end of 2023 was 28677, with a 61 GW total capacity, 114 TWh output and 22.2% portion in the gross power production [37]. From these statistics, we can see the crucial role of onshore wind energy in Germany, and thus the importance of onshore wind energy calculations which is a big part of this project. Therefore, the overall task of the wind team, which is improving the accuracy of the onshore wind energy calculations, will contribute positively for the achievement of the goals of this project

Solar Energy usage by a solar panel (Focus on diffused solar radiation)

Han Lin Aung 22201576, Charles Albert Watiemeh Cobb 22307151, Ye Yint Myint Htwe 22209692, Kaung Htet San 22201794.

Data

The data used to establish the diffuse models were obtained primarily from DWD and NOAA, as well as information from the previous semesters' work done on this project.

The Global Horizontal Sunlight and Diffuse Horizontal Irradiance values were obtained from the DWD in J/cm². It was then converted into W/m². Using the Solar Zenith Angle obtained for each location from NOAA, the Direct Horizontal Irradiance was also obtained. The Solar Azimuth Angles were also obtained from NOAA. Together with this information, well-known constants such as the Solar Constant of 1361 W/m² [9], some mathematical computations using Matlab and formulae found from the various models as listed below, the various diffuse irradiances were found for each location in a 10-min resolution. The results were put together and a choice of the best model to use was obtained. Aside from research papers that indicated the superiority of the Perez Model, which is the latest model of the diffuse irradiance models, graphs and correlation methods were used to determine the model to be used.

Global Horizontal Irradiance (GHI)

Global horizontal irradiance is the total solar radiation per unit area measured at a horizontal surface on the earth. It is typically presented in W/m² and can be broken down into two components: direct normal irradiance (DNI) and diffuse horizontal irradiance (DHI). The relationship between GHI, DHI and DNI is expressed in the equation below: [3]

$$\text{GHI} = \text{DHI} + \text{DNI} * \cos(\theta_{\text{zenith}})$$

Units: W/m²

Direct Normal Irradiance (DNI)

Direct normal irradiance (DNI) is the portion of solar radiation that reaches the earth on a direct path from the sun. [3]

Units: W/m²

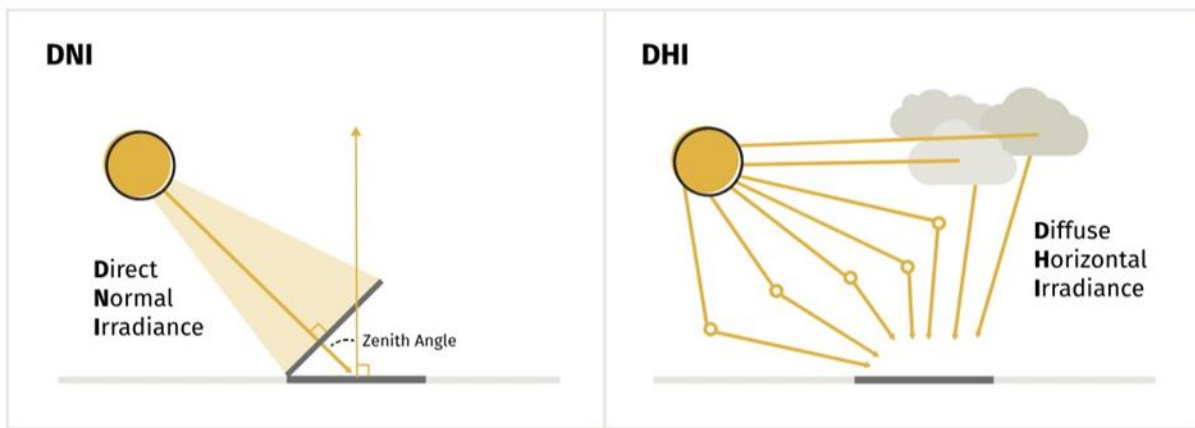


Fig 1: Direct Normal Irradiance and Diffuse Horizontal Irradiance [3]

Diffuse Horizontal Irradiance (DHI)

Diffuse horizontal irradiance (DIF or DHI) is the portion of solar radiation that reaches the earth indirectly. Water vapour, aerosols and clouds reflect and absorb solar radiation, diffusing it throughout the atmosphere. [3]

Units: W/m^2

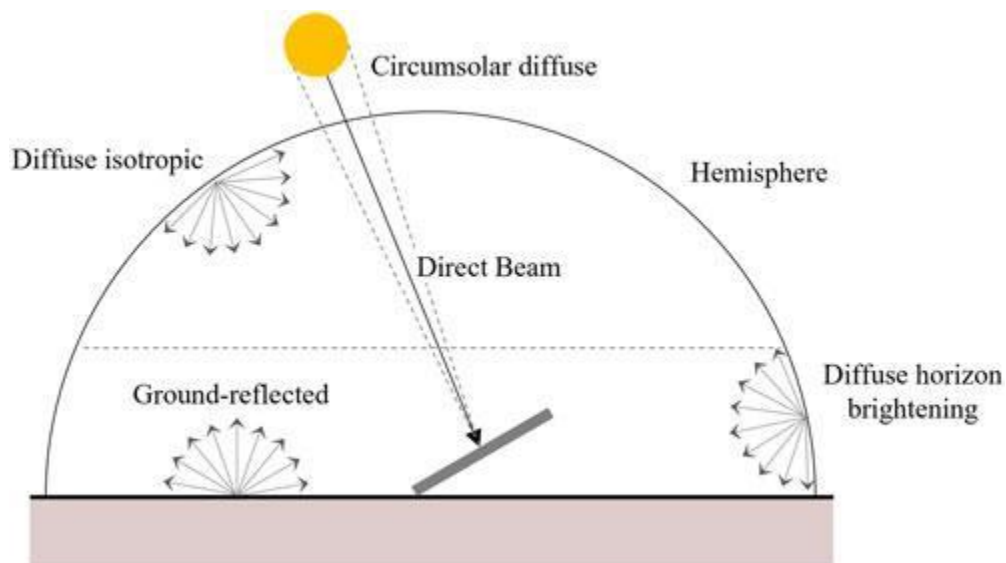


Fig 2: Different solar contributions to global tilted irradiance.

Overview of the Hay and Davies Model

The Hay and Davies model improves upon simpler isotropic models by accounting for the anisotropic nature of diffuse radiation. It divides the sky diffuse irradiance into two components [4]:

1. Circumsolar Component: Diffuse radiation concentrated near the sun's position.
2. Isotropic Background Component: Uniformly distributed diffuse radiation across the sky.

The model calculates the diffuse solar radiation on a tilted surface as follows:

$$E_d = DHI \times \left[A_i R_b + (1 - A_i) \frac{1 + \cos(\theta_T)}{2} \right] \quad [4]$$

Where:

DHI = Diffuse horizontal irradiance

A_i = Anisotropy index, representing the circumsolar component

R_b = Geometric factor, the ratio of beam radiation on the tilted surface to that on a horizontal surface

θ_T = Tilt angle of the solar panel

Derivation and Explanation of Parameters

1. Anisotropy Index (A_i)

The anisotropy index (A_i) represents the portion of diffuse radiation that is concentrated near the sun's position (circumsolar component). It is calculated as:

Where: $A_i = \frac{DNI}{E_a}$

DNI = Direct Normal Irradiance

E_a = Extraterrestrial solar radiation (you can see the detailed calculation for E_a in Perez Model Calculation Part)

2. Geometric Factor (R_b)

The geometric factor (R_b) accounts for the geometric relationship between the tilted surface and the sun's position. It is calculated as:

Where: $R_b = \frac{\cos(AOI)}{\cos(\theta_z)}$

AOI = Angle of incidence of the sun's rays on the tilted surface

θ_z = Solar zenith angle (angle between the sun's rays and the vertical)

R_b adjusts the beam radiation component for the tilt of the surface

3. Isotropic Background Component

The term $(1 - A_i)$ represents the portion of diffuse radiation that is isotropic (uniformly distributed across the sky). The factor $\frac{1 + \cos(\theta_T)}{2}$ counts for the view factor of the tilted surface to the sky, representing the fraction of the sky "seen" by the tilted surface [5].

4. Combining Components

The total diffuse radiation on the tilted surface is the sum of:

- The circumsolar component: $A_i R_b DHI$
- The isotropic background component: $(1 - A_i) \frac{1 + \cos(\theta_T)}{2} DHI$

Reindl Sky Diffuse Model

The second method analysed in this project is the Reindl Sky Diffuse model. This model is an anisotropic diffuse irradiance model designed to estimate the diffuse solar radiation incident on a tilted surface. It improves upon isotropic models by incorporating additional components like circumsolar and horizon brightening, making it more accurate for varied atmospheric conditions [6].

This model is widely used in photovoltaic (PV) performance studies due to its simplicity and relatively good accuracy. The Reindl model divides the diffuse irradiance into three main components: 1. The isotropic component assumes a uniform distribution of diffuse radiation across the sky dome. 2. The circumsolar component accounts for the concentration of diffuse light in the region near the sun. 3. Horizon brightening reflects the increased brightness near the horizon.

Formula Derivation and Parameter Explanation

The total sky diffuse radiation on the tilted surface using Reindl's model formulation is:

$$E_d = DHI \cdot [A_i \cdot R_b + (1 - A_i) \cdot (1 + \sqrt{\sin \gamma})^3]$$

Where:

E_d = diffuse irradiance on the tilted surface DHI = Diffuse horizontal irradiance DNI = Direct normal irradiance GHI = Global horizontal irradiance A_i = Anisotropy index β = Tilt angle of the array γ = Azimuth angle R_b = Geometric factor

Explanation of Horizon Brightening

This model extends the Hay and Davies model by an additional factor to the "brightening" term " $DHI \cdot (1 - A_i) \cdot \sin^3 \gamma$ " to account for horizon brightening. It accounts for the enhanced brightness near the horizon with a stronger effect for smaller tilt angles [6].

The Reindl model provides a balance between computational simplicity and accuracy, making it well-suited for estimating diffuse irradiance on tilted surfaces [6], [7]. It incorporates empirical parameters to account for circumsolar and horizon effects, which are essential for the realistic representation of diffuse solar

radiation [7], [8]. This model is particularly useful for preliminary solar energy analyses and PV system design.

Perez Sky Diffuse Model

The estimation of diffuse radiation on a tilted surface is calculated by the Perez Sky Diffuse Model which is comprised of three different components: [16]

- (a) Isotropic,
- (b) Circumsolar,
- (c) Horizon Brightening

Isotropic Sky Diffuse Model

The simplest approach to modelling the sky diffuse irradiance is to assume isotropic conditions, i.e., uniform distribution of the diffuse radiance from the sky hemisphere. Due to the assumption of uniform radiance, the isotropic sky diffuse transposition factor only depends on the slope of the tilted plane, θ_T . [9]

$$R_d = \frac{1 + \cos(\theta_T)}{2}$$

The in-plane diffuse sky irradiance, D_c can be calculated as: [9]

$$D_c = DHI * R_d$$

θ_T = the tilt angle of the array

DHI = diffuse horizontal irradiance

The two main anisotropic effects are (1) enhanced irradiance from the sun region due to forward scattering (circumsolar) and (2) horizon zenith gradients that include horizon brightening in clear conditions and zenith brightening in heavily overcast conditions. [10]

Circumsolar And Horizon Brightening Component

Angle of Incidence (AOI)

The angle of incidence (AOI) between the Sun's rays and the PV array can be determined as:

$$AOI = \cos^{-1} [\cos(\theta_{zenith}) \cos(\theta_T) + \sin(\theta_{zenith}) \sin(\theta_T) \cos(\theta_A - \theta_{A,array})]$$

Where,

θ_{zenith} = Zenith angle

θ_T = the tilt angle of the array

θ_A = Azimuth angle

$\theta_{A,array}$ = Azimuth angle of the array

The azimuth angle convention is defined as degrees east of north (e.g. North = 0, East = 90, West = 270). Array azimuth is defined as the horizontal normal vector from the array surface. An array facing south has an array azimuth of 180°. Array tilt is defined as the angle from horizontal. [11]

$$a = \max [0, \cos(AOI)]$$

$$b = \max [\cos 85^\circ, \cos(\theta_{zenith})]$$

The terms a and b represent the cosine of circumsolar incidence angles on the considered tilted plane and the horizontal, respectively. [9]

The 1986 formulation of the Perez model treated the circumsolar irradiance as a uniform contribution from a circular region centred around the sun (15° half-angle) and horizon brightening from a finite band at the horizon (angular thickness of 6.5°).[12] The sky conditions were discretised into 200 categories based on the zenith angle (θ_{zenith}), diffuse horizontal irradiance (DHI), and sky clearness (ϵ_0) is defined as [9]:

$$\epsilon_0 = \frac{DHI + DNI}{DHI} \quad (1986 \text{ and } 1987 \text{ models})$$

DNI = direct normal irradiance

The Perez 1987 model featured several improvements, including the simplification of the governing equation and allowing negative horizon brightening coefficients (equivalent to brightening at the top of the sky dome) [10]. The new model formulation had the advantage of reducing the number of empirical coefficients from 480 to only 48. Additionally, the horizon brightening region was modified to be an infinitesimally thin region at 0-degree elevation, and the sky condition parameterisation was adjusted by replacing DHI with sky brightness (Δ) as defined by: [9]

$$\Delta = \frac{DHI * m}{I_o}$$

where,

m = relative airmass

I_o = extraterrestrial normal irradiance

Air Mass (m)

Air mass is a relative measure of the optical length of the atmosphere. At sea level, when the sun is directly overhead (zenith angle = 0), the air mass is equal to 1. As the zenith angle becomes larger, the path of direct sunlight through the atmosphere grows longer and air mass increases. “Relative” air mass is only a function of the sun's position and assumes the observer is at sea level.

The simplest estimate of relative air mass (ignoring land elevation effects) assumes a spherical earth and atmosphere. The relative air mass is simply a trigonometric function of the zenith angle:

$$m = \frac{1}{\cos(\theta_{\text{zenith}})}$$

This simple approximation is quite accurate for zenith angles less than 80 degrees. When the sun is near the horizon, more complex and accurate models are warranted. [13]

Extraterrestrial normal irradiance (I_0)

In addition to the geometric (space) and chronologic (time) relationships between collector surfaces on Earth and the sun, the intensity of the sun's irradiation changes based on the distance between the Earth and the sun because of the elliptical orbit of the Earth. While the average solar radiation flux is 1361 W/m^2 , the annual fluctuation due to the Earth's elliptical orbit is greater than $\pm 40 \text{ W/m}^2$, resulting in peak extraterrestrial radiation flux of about 1410 W/m^2 in January and lows of about 1320 W/m^2 in June. The following equation can be used to represent extraterrestrial normal irradiance, I_0 . [14]

$$I_0 = \text{solar constant} * (1 + 0.033 \cos(\frac{360 * n}{365}))$$

I_0 = extraterrestrial normal irradiance on the n^{th} day of the year

solar constant = 1361 W/m^2

n = the day of the year (1 for January 1st, 365 for December 31st)

Furthermore, the 1988 and 1990 versions featured several improvements and simplifications, including modelling circumsolar irradiance as a point source instead of an arbitrarily sized region around the sun disk [15,17]. This has the associated benefit that it is no longer necessary to solve non-linear equations when deriving coefficients for the new model formulation. The sky clearness was also modified to eliminate its dependence on the solar zenith angle. The zenith independent sky clearness (ϵ) is expressed as: [9]

$$\epsilon = [\epsilon_0 + \kappa (\theta_{\text{zenith}})^3] / [1 + \kappa (\theta_{\text{zenith}})^3] \quad (\text{zenith independent, 1988 and 1990 models})$$

κ = constant equal to 1.041 when θ_{zenith} is in radians

ϵ bin	F_{11}	F_{12}	F_{13}	F_{21}	F_{22}	F_{23}
[1.000–1.065)	−0.008	0.588	−0.062	−0.060	0.072	−0.022
[1.065–1.230)	0.130	0.683	−0.151	−0.019	0.066	−0.029
[1.230–1.500)	0.330	0.487	−0.221	0.055	−0.064	−0.026
[1.500–1.950)	0.568	0.187	−0.295	0.109	−0.152	−0.014
[1.950–2.800)	0.873	−0.392	−0.362	0.226	−0.462	0.001
[2.800–4.500)	1.132	−1.237	−0.412	0.288	−0.823	0.056
[4.500–6.200)	1.060	−1.600	−0.359	0.264	−1.127	0.131
[6.200–∞]	0.678	−0.327	−0.250	0.156	−1.377	0.251

Table: Perez model coefficients (all sites composite 1990) [15].

The anisotropic coefficients F_1 and F_2 are functions of the sky conditions and express the degree of circumsolar and horizon/zenith anisotropy, respectively. [9]

$$F_1 = \max [0, F_{11}(\epsilon) + \Delta \cdot F_{12}(\epsilon) + \theta_{\text{zenith}} F_{13}(\epsilon)]$$

$$F_2 = F_{21}(\epsilon) + \Delta \cdot F_{22}(\epsilon) + \theta_{\text{zenith}} F_{23}(\epsilon)$$

bin	Lower Bound	Upper Bound
1 Overcast	1	1.065
2	1.065	1.230
3	1.230	1.500
4	1.500	1.950
5	1.950	2.800
6	2.800	4.500
7	4.500	6.200
8 Clear	6.200	—

Table 1: Sky clearness bins [15].

The mathematical formulation of the 1990 Perez et al. [15] sky diffuse transposition factor, R_d is provided as: [9]

$$R_d = (1-F_1) \frac{1+\cos(\theta_T)}{2} + F_1 \frac{a}{b} + F_2 \sin(\theta_T)$$

Therefore, diffuse sky irradiance, D_c can be calculated as:

$$D_c = DHI * R_d$$

Selection

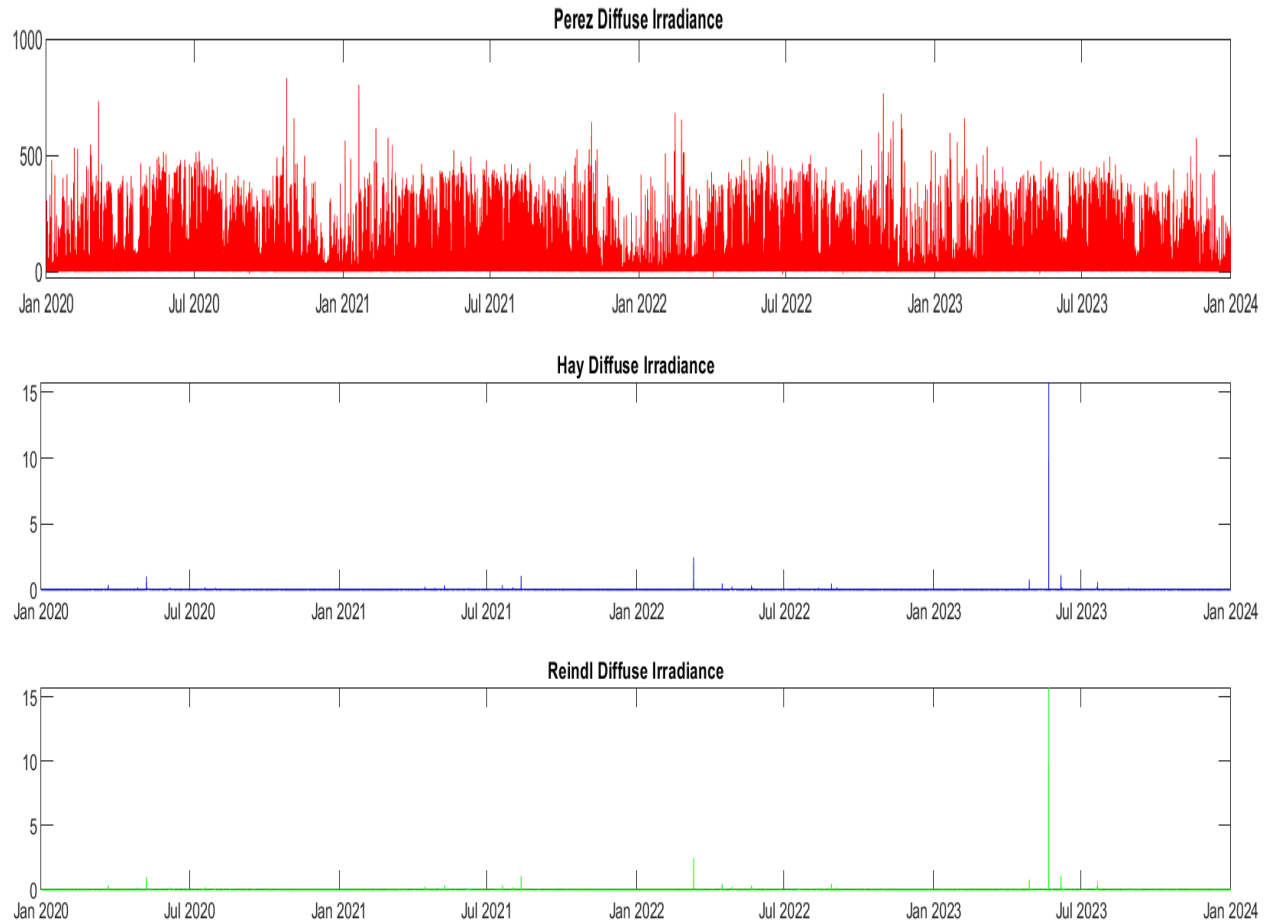
From the above-mentioned models, the Perez model was selected as the most suitable. Apart from the fact that the Perez model is the most recent diffuse model, using three diffuse models resulted in output which further informed the decision to choose the Perez model. Using Pearson's correlation between the three diffuse models, Perez, Reindl, and Hay and Davies, we had the following results:

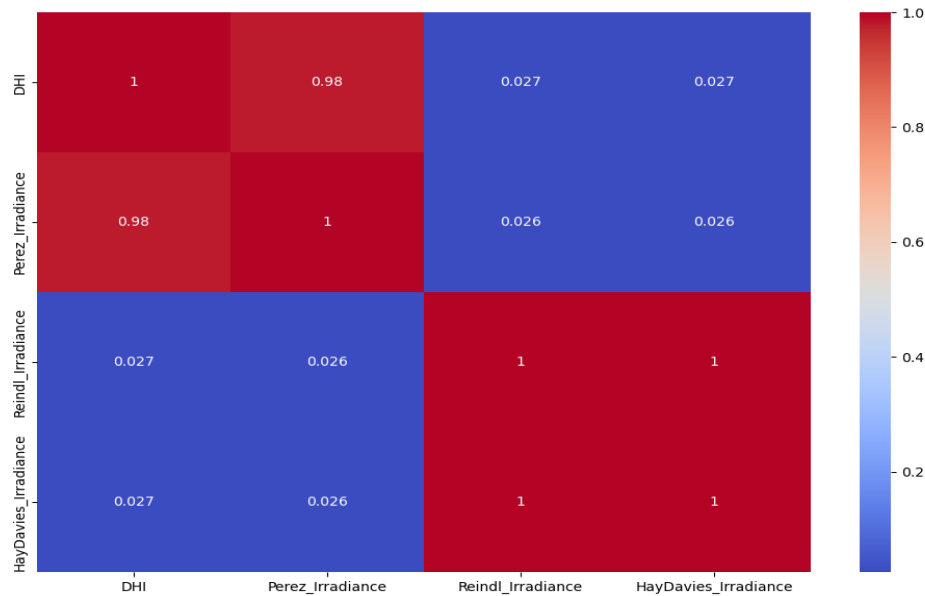
Correlation between Perez and Reindl models: 0.0260

Correlation between Perez and Hay models: 0.0258

Correlation between Reindl and Hay models: 1.0000

In view of this, as well as the graphs of the models, the selection for Perez model was made. The results from the Perez model had all values to be more practical compared with the other two models that have outliers that are magnitudes($1e5$) off realistic obtainable values.

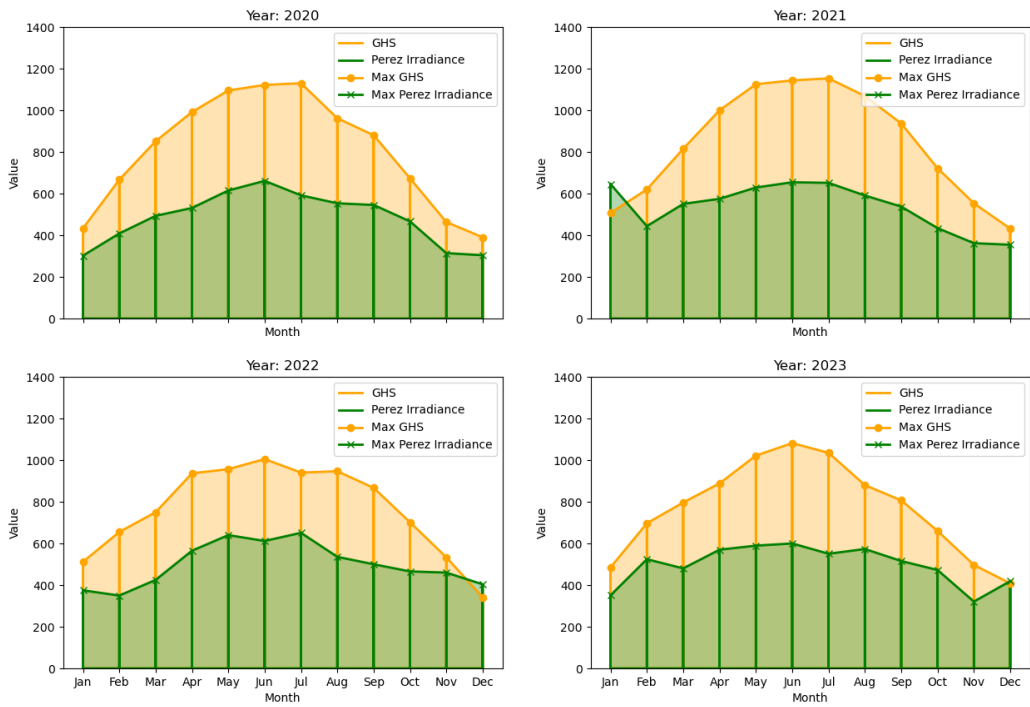




The heatmap illustrates the correlation between the three models and their ability to estimate diffuse horizontal irradiance (DHI), with DHI values sourced from the DWD (Deutscher Wetterdienst) website as the reference. The analysis reveals that the Perez model demonstrates a remarkably strong correlation (0.98) with the DWD's DHI values, indicating its high accuracy in capturing diffuse irradiance. In contrast, the other two models show significantly weaker correlations, with values of 0.027 and 0.026, suggesting limited reliability in estimating diffuse radiation. Given these findings, the Perez model was selected for further analysis due to its superior alignment with observed DHI data.

Using the Perez model, we were able to obtain the graph of the average Global Horizontal Irradiance and the Perez Irradiance.

Annual Comparison of Global Horizontal Irradiance (GHI) and Perez Irradiance in Mühldorf



Wind Team: Improvement of the Mathematical Wind Profile Model

Ahmed Fouad 22309260, Amos Addai Worae 22208595, Ahmed Saafan 22300578, Asrin Nakipoglu 22308182.

Previous Work, Goal of the Team

Following the work of the previous project groups, the goal of the wind team was determined to be the improvement of the mathematical wind profile model used in the calculations. This model is used for the determination of wind speed, which is a key variable that affects the accuracy of power and energy calculations of the wind turbine.

For the calculation of the output power of the wind turbine, the previous project groups had come up with a logarithmic function that requires the wind speed as a variable. This wind speed value has to be taken at the hub height of the wind turbine, since the wind enters the wind turbine at this height. The hub height of the wind turbine selected for this project is 199 m, but the anemometers that measure wind speed at weather stations are positioned at much lower heights than this [18]. Thus, the speed at the hub height has to be estimated from the speed measured at the anemometer height. For this purpose, a mathematical wind profile model is required.

Up to this point of the project, the logarithmic wind profile model had been used for the estimation of the wind speed. This is a popular and simple mathematical model used widely, given on equation 1 [19, 20].

$$v_{h_2} = v_{h_1} \times \frac{\ln \left(\frac{h_2}{z_0} \right)}{\ln \left(\frac{h_1}{z_0} \right)} \quad (1)$$

Where

- h_1 : Reference height
- h_2 : Height of estimation
- v_{h_1} : Wind speed at the reference height
- v_{h_2} : Estimated wind speed
- z_0 : Roughness length

The work of the previous project groups had focused solely on $h_2 = 200 \text{ m}$ which is the hub height of the turbine rounded to the nearest hundred, and $z_0 = 0.4 \text{ m}$ which corresponds to “Towns, villages, agricultural land with many or high hedges, forests and very rough and uneven terrain” [20]. However, according to the project supervisor, Professor Mátéfi-Tempfli, this mathematical model overestimated the wind speed and thus improvement was required.

Observing the Inaccurate Estimation of the Log Law

As a first step, an exercise was conducted to observe this inaccurate estimation. The aim was to calculate wind speeds for a chosen weather station using the logarithmic model, and to compare the results with wind speeds obtained by DWD, Deutsche Wetterdienst. The wind speed data of DWD is found in the form of a report for each weather station, for a specific period of time. This data is represented in the report by the following four parameters, given for the roughness lengths of 0 m, 0.003 m, 0.1 m and 0.4 m, and the heights of 10 m, 25 m, 50 m, 100 m and 200 m [21, 22, 23]:

- Weibull A*: Weibull scale parameter [m/s]
- Weibull k*: Weibull shape parameter

$\bar{v}$:	Mean wind speed [m/s]
$\bar{p}$:	Mean wind energy potential [W/m ²]

As the main focus of the wind team was the improvement of the wind estimation, only the two Weibull parameters and $\bar{v}$ were considered. The two Weibull parameters come from the Weibull distribution, which is a widely utilized continuous probability distribution [21,23]. These two parameters represent the probability density function of the Weibull distribution of all the wind speed data within the time period of the DWD report.

The required measured wind speed data was obtained from CDC, the Climate Data Center. CDC is an open access database that has recent and historical wind speed data for different measurement intervals such as daily, hourly and 10-minutes [34]. Continuing with the time interval used by the previous project groups, the 10-minutes measurement interval was chosen.

The weather station for this exercise was chosen according to the availability of wind speed data. The reasoning behind this is the fact that DWD reports have been created for specific time periods such as 1992-2004 [22], and the wind speed data on CDC has limited availability in terms of time [34]. Therefore, a station that has all the wind speed data available for the time period of the DWD report was researched, and the station Bremen (station code 01691) was chosen eventually [34]. The Bremen weather station has an anemometer height of 10 meters, and the DWD report for the station is for the time period 01.01.1995-31.12.2004.

The measured wind speed data was obtained from CDC and compiled into an Excel spreadsheet [34]. Then, the data was cleaned from faulty measurements. For the height values 25 m, 50 m, 100 m and 200m, and the roughness values 0.03 m, 0.1 m and 0.4 m, the logarithmic wind profile, equation 1, was applied to each measured datapoint in the spreadsheet. The roughness length of 0 m was not considered as it causes a division by zero in the calculations.

The next step was converting the calculated values into respective Weibull parameters A and k , and the mean wind speed $\bar{v}$. For this, the Weibull Calculator of The Swiss Wind Power Data Website was used [23]. This tool requires the frequencies of wind speed within 1 m/s intervals from 0 m/s to 20 m/s as input [23]. Using these values, the tool fits a Weibull curve representing the wind speed distribution and calculates the *Weibull A*, *Weibull k* and $\bar{v}$ values. The required frequency values were calculated using Excel, and the three parameters were obtained from the tool for each height and roughness pair. The Weibull curves fit by the tool for 200m height are given on figures 3-6.

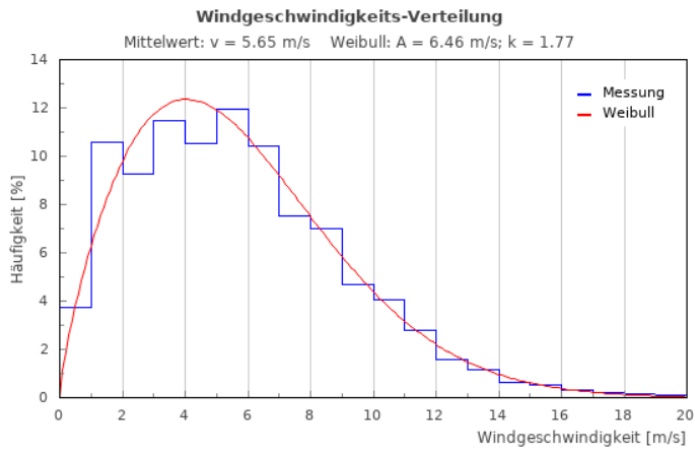


Figure 3: Weibull curve for 0.03 m roughness length at 200 m height

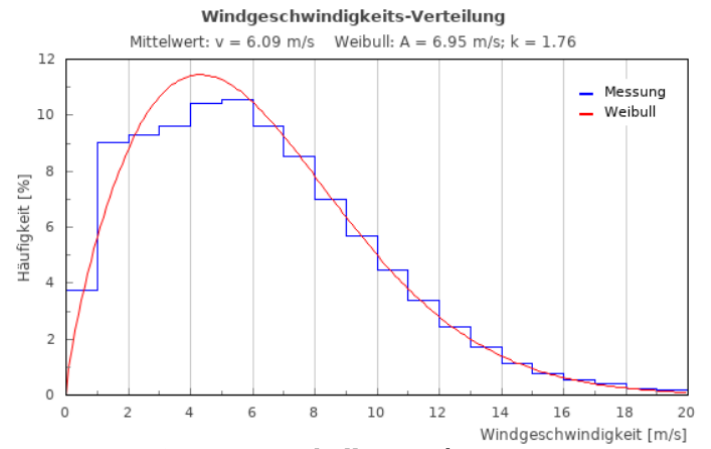


Figure 4: Weibull curve for 0.1 m roughness length at 200 m height

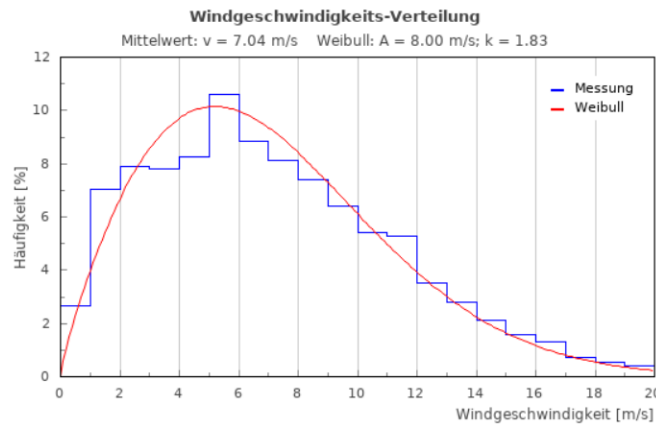


Figure 5: Weibull curve for 0.4 m roughness length at 200 m height

To verify the parameters obtained from the tool, they were re-calculated using the ‘wblfit’ and ‘mean’ functions of MATLAB [24]. The obtained parameters from both tools are given on table 2. It was observed that the A , and $\bar{v}$ values obtained from MATLAB are larger compared to values obtained from the Swiss Website. It was concluded that the reason for this is the 20 m/s limitation of the Swiss Website tool, as it excludes the higher wind speed values.

Table 2: A , k and $\bar{v}$ parameters obtained from the Swiss Website and MATLAB

When the obtained parameters were compared to the parameters of the DWD report, the following observations were made:

- For the 0.03 m and 0.1 m roughness lengths, the logarithmic model underestimates the A and $\bar{v}$ parameters.
- For the 0.4 m roughness length, the logarithmic model overestimates the A and $\bar{v}$ parameters.
- The obtained k parameters do not vary significantly with height and roughness length. However, the k values are higher in the DWD report, and tend to increase with roughness length.

Height	Parameter	Used Tool	Roughness			
			0 m	0.03 m	0.1 m	0.4m
10m	<i>Weibull A</i> (m/s)	Swiss Website MATLAB	- -	4.21 4.24	4.21 4.24	4.21 4.24
	<i>Weibull k</i>	Swiss Website MATLAB	- -	1.71 1.71	1.71 1.71	1.71 1.71
	$\bar{v}$ (m/s)	Swiss Website MATLAB	- -	3.69 3.78	3.69 3.78	3.69 3.78
25m	<i>Weibull A</i> (m/s)	Swiss Website MATLAB	- -	4.93 4.91	5.11 5.08	5.49 5.45
	<i>Weibull k</i>	Swiss Website MATLAB	- -	1.73 1.71	1.74 1.71	1.75 1.71
	$\bar{v}$ (m/s)	Swiss Website MATLAB	- -	4.33 4.38	4.46 4.53	4.80 4.86
50m	<i>Weibull A</i> (m/s)	Swiss Website MATLAB	- -	5.40 5.42	5.67 5.72	6.40 6.36
	<i>Weibull k</i>	Swiss Website MATLAB	- -	1.7 1.71	1.7 1.71	1.76 1.71
	$\bar{v}$ (m/s)	Swiss website MATLAB	- -	4.77 4.83	5.03 5.1	5.60 5.67
100m	<i>Weibull A</i> (m/s)	Swiss Website MATLAB	- -	5.98 5.92	6.32 6.36	7.29 7.27
	<i>Weibull k</i>	Swiss Website MATLAB	- -	1.76 1.71	1.73 1.71	1.83 1.71
	$\bar{v}$ (m/s)	Swiss Website MATLAB	- -	5.21 5.28	5.56 5.67	6.34 6.49
200m	<i>Weibull A</i> (m/s)	Swiss Website MATLAB	- -	6.46 6.43	6.95 6.999	8.00 8.19
	<i>Weibull k</i>	Swiss Website MATLAB	- -	1.77 1.71	1.76 1.71	1.83 1.71
	$\bar{v}$ (m/s)	Swiss Website MATLAB	- -	5.65 5.73	6.09 6.24	7.04 7.3

Height	Parameter	Used Model	
		Logarithmic	Power
200m	Weibull A (m/s)	8.19	10.11
	Weibull k	1.71	1.71
	$\bar{v}$ (m/s)	7.3	9.01

- Since 10 m is the height at which the wind measurements were taken, it has identical values for every roughness length on table 2. However, the DWD report has decreasing A and $\bar{v}$ parameters with increasing roughness length.

These observations showed that:

- In the DWD report, A and $\bar{v}$ parameters were estimated for different roughness lengths for a constant height value. Since this is beyond the capabilities of the logarithmic model, this manner of comparing parameters for different height-roughness length pairs might not result in the most reliable conclusions regarding the accuracy of the mathematical model.
- Regardless of the unreliability of the conclusions, the logarithmic model results in inaccurate parameter estimates since parameters for none of the roughness lengths are acceptably close to the DWD report.

Therefore, the necessity of finding a more accurate mathematical model and investigating further regarding roughness lengths was made clear.

Power Law

Upon research, the Power Law, another widely used mathematical wind profile model, was found. The Power Law is characterized by equation 2 [25].

$$v_{h_2} = v_{h_1} \times \left(\frac{h_2}{h_1}\right)^a \quad (2)$$

Where a is the wind profile/wind shear exponent that depends on the terrain type and atmospheric stability [25, 26]. When the implementation of the model was attempted, it was observed that it is a challenge to obtain the variable a for respective terrain types. After further research, a reference was found which is given in figure 6 [26].

Area type	a
Smooth hard ground, calm water	0.10
Short grass on untilled ground	0.14
Level country with foot-high grass, occasional tree	0.16
Tall row crops, hedges, a few trees	0.20
Many trees and occasional buildings	0.22 – 0.24
Wooded country – small towns and suburbs	0.28 – 0.30
Urban areas with tall buildings	0.4

Figure 6: Wind profile exponents for different terrain types [26]

For an accurate comparison with the results obtained from the Logarithmic Law during the initial exercise, a was chosen as 0.29 as it roughly corresponds to the used roughness length of 0.4. Using Excel and MATLAB, equation 2 was applied to the wind speed data from the Bremen station at $h_2=200$ m, and the parameters A , k and $\bar{v}$ were obtained. The results are given on table 3, comparatively.

Table 3: Obtained parameters at 200 m for Logarithmic and Power Laws

It was observed that the overestimation of the A and $\bar{v}$ parameters is even larger compared to the Logarithmic wind profile. Due to this result, equation 2 was deemed unfit for use as the mathematical model.

Surface Roughness

Upon investigation of the DWD report, it was found out that each weather station has a ‘roughness rose’ with 12 directional sectors, each covering 30°. This map-like figure describes the distribution of different roughness values around the anemometer. The ‘roughness rose’ is accompanied by data indicating the different roughness values within the sectors, and a single reference roughness length value for each sector [22]. These findings about the roughness length at a weather station showed that applying any mathematical wind profile model for random roughness lengths and comparing to the DWD report is not realistic. This is due to the fact that the considered mathematical models can only estimate wind speeds between different heights, not between different roughness lengths. Therefore, station-specific roughness lengths obtained from the DWD report were decided to be used in the application of the mathematical models.

First, the wind data on the Excel spreadsheet used in the exercise was filtered according to the directional sectors of the wind measurements, and the roughness length values in the Logarithmic law equation were substituted with the reference roughness lengths of the different sectors. These reference roughness lengths for the Bremen station are given on table 4, where $z_{0,ref}$ is the reference roughness length [22]. The A , k and $\bar{v}$ parameters were obtained using MATLAB, which are given on table 5.

Wind Direction	$z_{0,ref}$ (m)	Wind Direction	$z_{0,ref}$ (m)
345°-15°	0.203	165°-195°	0.091
15°-45°	0.170	195°-225°	0.081
45°-75°	0.131	225°-255°	0.083
75°-105°	0.121	255°-285°	0.091
105°-135°	0.075	285°-315°	0.092
135°-165°	0.093	315°-345°	0.135

Table 4: Wind direction sectors and corresponding reference roughness lengths

Next, following the suggestion of the project supervisor Professor Mátéfi-Tempfli, the weighted mean of the reference roughness lengths was calculated. For this, the number of measured datapoints in each sector was obtained, frequency values corresponding to each sector were calculated (table 6), and these frequency values were used as weights and input into an online tool. The weighted mean roughness length of the Bremen station was obtained from the tool as 0.1011313 m [27]. The A , k and $\bar{v}$ parameters were also obtained for this weighted mean roughness length value, given on table 5.

Different $z_{0,ref}$ values				Weighted Mean $z_{0,ref}$ value			
Height	Weibull A (m/s)	Weibull k	$\bar{v}$ (m/s)	Height	Weibull A (m/s)	Weibull k	$\bar{v}$ (m/s)
200m	6.97	1.698	6.22	200m	6.99	1.691	6.24

Table 5: Parameters for the sector-specific and weighted mean roughness lengths

Wind Direction	Number of Datapoints	Frequency (%)	Wind Direction	Number of Datapoints	Frequency (%)
----------------	----------------------	---------------	----------------	----------------------	---------------

Height (m)	Direction	Weibull A (m/s)		Weibull k		$\bar{v}$ (m/s)	
		Reference	Calculated	Reference	Calculated	Reference	Calculated
10	15°-45°	4.3216618	4.233353344	1.94	1.95	2.25	2.25
25	45°-75°	5.3422044	5.074448256	2.07	2.25	2.55	2.55
50	75°-105°	6.2646703	5.719424193	2.28	2.55	2.85	2.85
100	105°-135°	7.4566640	6.351344728	2.48	2.85	3.15	3.15
200	135°-165°	9.236402	6.997345555	2.38	3.15	3.45	3.45

Table 6: Values calculated and used for obtaining the weighted mean roughness length

It was observed that the weighted mean is a good approximation of all the different roughness lengths as the parameters are very close.

Modified Power Law [28].

Following further research, a modified version of the previously investigated Power Law was found. According to this paper, the wind profile exponent, a , can be calculated in a way that considers the wind speed at a reference height and the surface roughness [28]. This was deemed as a highly advantageous aspect of this law since it solves the challenge of obtaining the a value for different terrain types and improves the accuracy of the wind profile model itself by considering these area-specific variables. Taking h_1 as the aforementioned reference height, the equation of the wind profile exponent is given as [28]:

$$a = \left(\frac{z_0}{h_1}\right)^{0.2} \times (1 - [0.55 \times \log(v_1)]) \quad (3)$$

Inserting equation 3 into equation 2, the overall equation of the mathematical model becomes [28]:

$$v_{h_2} = v_{h_1} \times \left(\frac{h_2}{h_1}\right)^{\left(\frac{z_0}{h_1}\right)^{0.2} \times (1 - [0.55 \times \log(v_1)])} \quad (4)$$

The law was implemented to the wind speed data of Bremen on Excel, for the weighted mean roughness value of 0.101 m. This was done by applying equation 3 and inserting the result into equation 2 for each wind speed measurement. Then, the three parameters were obtained using MATLAB. The results are given alongside the reference values from the DWD report on table 7 [22]. It should be noted that the parameters at 10 m height are not calculated but measured.

Table 7: Calculated and Reference Parameters for the Modified Power Law

It can be seen that the parameters obtained using the Modified Power Law are significantly close to the ones from the DWD report.

As the next step, the model was compared to the reference using the least squares method, following the recommendation of the project supervisor Professor Mátéfi-Tempfli [29]. The reference parameters were fit into curves using MATLAB, and the calculated parameters from the Modified Power Law were scattered onto the same plot. To have a comparative view, the parameters were also calculated using the Logarithmic law for the same heights and the weighted mean roughness length, which are given on table 8 [22]. The calculated parameters from this law were also scattered onto the same plot to visualize the difference in accuracy between the two laws. The plots are given on figures A.1-A.3 in the appendix.

Table 8: Calculated and Reference Parameters for the Logarithmic Law

	Modified Power Law	Log Law
<i>RMSE Weibull A (m/s)</i>	0.18	1.27
<i>RMSE Weibull k</i>	0.22	0.63
<i>RMSE $\bar{v}$ (m/s)</i>	0.156	1.09

Table 9: Root Mean Square Errors for Both Laws

Finally, the root mean square error, RMSE, of both laws were calculated using equation 5, excluding the values at 10 m height [30].

	<i>Weibull A (m/s)</i>		<i>Weibull k</i>		<i>$\bar{v}$ (m/s)</i>	
Height (m)	Reference	Calculated	Reference	Calculated	Reference	Calculated
10	4.32	4.2405414	1.94	1.7075957	3.83	3.7812731
25	5.34	5.3874339	2.07	1.8709402	4.73	4.7814626
50	6.26	6.4569163	2.28	2.0168865	5.54	5.71935
100	7.45	7.7387061	2.48	2.1875289	6.61	6.8508611
200	9.2	9.2749494	2.38	2.3897152	8.15	8.2182354

$$RMSE = \sqrt{\frac{\sum (x_{i,clc} - x_{i,ref})^2}{n}} \quad (5)$$

Where $x_{i,clc}$ represents the calculated values, $x_{i,ref}$ represents the reference values, and n represents the number of values. The results are given on table 9.

- **Rostock**

To test the model at another weather station with different terrain, the same procedures were applied for the Rostock weather station (station code 04271). For this station, the DWD report is for the time period 01.01.1992-31.12.2004 [22]. The weighted mean roughness length was calculated as 0.0304 m, which is rather low due to the weather station being near a water body. The anemometer height at the station was found to be 22 m, which showed that different weather stations have different anemometer heights [22]. Alongside the reference parameters from the DWD report, the obtained parameters for the Modified Power Law and the Logarithmic law are given on tables 10 and 11, respectively [22]. The calculated RMSE values are given on table 12 [30]. The plots obtained from MATLAB using the least squares method are given on figures A.3-A.6 in the appendix.

Table 10: Calculated and Reference Parameters for the Modified Power Law**Table 11:** Calculated and Reference Parameters for the Logarithmic Law

			Modified Power Law		Log Law	
RMSE Weibull A (m/s)			0.675723		1.152454	
RMSE Weibull k (m/s)			0.180514		0.406341	
RMSE $\bar{v}$ (m/s)			0.645869		1.012173	
Height (m)	Reference	Calculated	Reference	Calculated	Reference	Calculated
10	4.943057	5.5319	1.8	1.73	4.396632	4.91
25	5.934844	6.01064	1.92	1.73	5.259015	5.3362
50	6.864844	6.6487	2.12	1.73	6.079015	5.8971
100	8.120673	7.281	2.25	1.73	7.19544	6.4579
200	10.04756	7.9133	2.18	1.73	8.893523	7.0187

Table 12: Root Mean Square Errors for Both Laws

Height (m)	Weibull A (m/s)		Weibull k		$\bar{v}$ (m/s)	
	Reference	Calculated	Reference	Calculated	Reference	Calculated
10	4.943057	5.5319	1.8	1.73	4.396632	4.91
25	5.934844	6.07	1.92	1.7979	5.259015	5.3775
50	6.864844	6.8608	2.12	1.8948	6.079015	6.065
100	8.120673	7.75	2.25	2.003	7.19544	7.75
200	10.04756	8.755	2.18	2.1238	8.893523	7.733

It was observed that compared to the Bremen station, there is a significant increase in the difference between the calculated and reference parameters and the RMSE values. However, it was noted that although the Modified Power Law underestimates the parameters for this weather station, it still shows an improvement over the Logarithmic Law.

It should be noted that at this point, it was recommended by the project supervisor that we opt for using the respective reference roughness length values for each directional sector instead of using a single weighted mean roughness length for each wind speed measurement. Implementation of this resulted in a slight increase in the overall RMSE value but a decrease in the error of the 200 m parameters. Therefore, the weighted roughness length was only used for comparison between the reference data from this point on.

Testing and Changes

- **Schmücke**

Following the observed improvements the Modified Power Law brings; it was decided to test it further. This time, a weather station with a high weighted mean roughness length was researched since it is the only end of the roughness spectrum that had not been tested yet. For this purpose, the Schmücke weather station (station code 04501) with a weighted mean roughness length of 0.3185 m was chosen [22]. The DWD report for this station is for the time period of 01.01.1993-31.12.2000; and the anemometer height of the station was found to be 24 m [22]. Both the Logarithmic and the Modified Power laws were applied to the measurement data, and the three parameters were obtained using MATLAB.

To compare the accuracy of the obtained parameters, some extra steps had to be taken since reference data for the 0.3185 m roughness length does not exist on the DWD report. The reference parameters for the five different heights were entered into the application GeoGebra as datapoints with roughness length as the x component and the parameter value as the y component [22, 31]. The different curve

fitting commands of GeoGebra were explored, and the FitExp command was found to be the suitable one for the datapoints at hand [32]. Using this command, five different exponential regression curves with respective functions in the form of $a \times e^{(bx)}$ were obtained for each parameter [32]. The curves obtained for *Weibull A* are given on figure 7, and the curves obtained for *Weibull k* and $\bar{v}$ are given in the appendix on figures A.7 and A.8, respectively [31]. From these curves, the required parameters corresponding to 0.3185 m roughness length were obtained.

When comparison of the parameters was attempted, it was observed that the reference parameters at the height of measurement, 24 m, are lower than the parameters obtained from MATLAB using the measurement data [22]. In an attempt to solve this problem, a similar approach was followed. This time, the reference parameters for the five different heights were entered into GeoGebra as datapoints with height as the x component, and the parameter value as the y component [22, 31]. Exponential curves were plotted again, and these curves were shifted along the y axis by the difference in the reference parameters and the parameters calculated by MATLAB [31]. Using the parameters obtained from these curves, the comparison was made, focusing solely on the parameters at 200 m height [22]. It was observed that the Modified Power Law performed really well, with error values around 0.1 m/s for the *Weibull A* and $\bar{v}$ parameters, whereas the Logarithmic Law resulted in errors close to 1 m/s.

However, these positive results were decided to be incorrect when the project supervisor warned about the inaccuracy of the aforementioned curve-shifting method and advised to investigate the reason behind the difference in parameters instead.

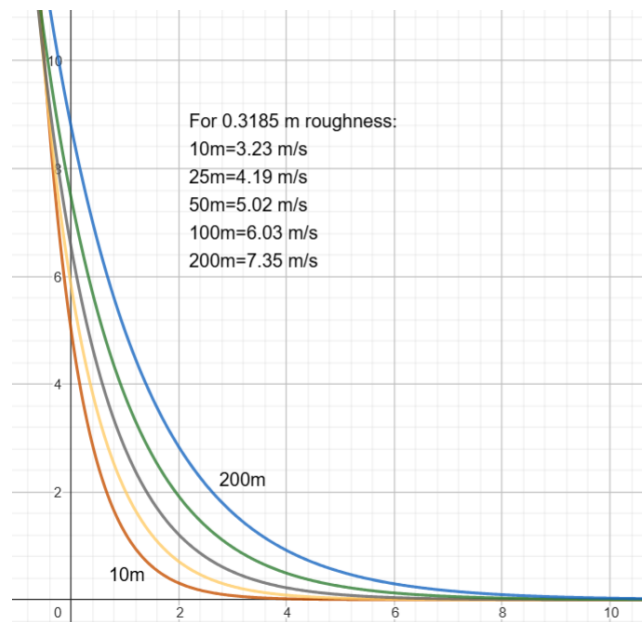


Figure 7: Exponential curves fit for the Weibull A parameter

- **Reasons behind the difference in measured parameters: Number of datapoints and Weibull curve intervals**

Following the advice of the project supervisor, any possible reasons behind the difference in reference and obtained parameters at the measurement height were searched. For this, the DWD reports, the method used for obtaining the three parameters from MATLAB, and the measured wind speed data obtained from CDC were investigated [22, 34]. As a result, it was concluded that the following might cause this difference:

- The number of measured datapoints in the Excel sheets are higher than the number of datapoints used in the DWD reports by factors close to 6 [22, 34].
- The reference parameters that had been used for comparison up to this point are not the measured ones, but the ones that were estimated by the mathematical model used by DWD. The parameters corresponding to the actual measured data are given in a different part of the DWD report [22].
- The bin intervals of the Weibull curves representing the wind speed data in the DWD report are 1 m/s [22]. When using the wblfit function of MATLAB to calculate the Weibull parameters, these bin intervals are unspecified and likely not 1 m/s [24].

To verify if these are indeed the reasons behind the difference, the following were done:

- Instead of wind speed data with 10-minute measurement intervals, hourly measurement data was obtained from CDC for the three weather stations considered up to this point [34]. The number of datapoints were compared to the number of datapoints from the DWD report and verified to be nearly identical [22,34].
- Alternatives to the wblfit function were researched, and the Distribution Fitter add-on of MATLAB was found [33]. Using this application and the hourly measurement data, the Weibull curves for the weather stations were plotted for 1 m/s bin intervals, and the Weibull parameters were obtained [33]. The parameters were also obtained using the wblfit function and compared with the parameters from the Distribution Fitter.
- The parameters were also compared with the parameters of the wind speed data with 10-minute measurement intervals.

Following these procedures, it was concluded that:

- The Distribution Fitter add-on and the wblfit function of MATLAB produced identical Weibull parameters, showing that the bin intervals do not affect the Weibull parameters.
- Using the hourly data for obtaining the parameters and comparing with the correct parameters in the DWD report removes the difference at the measured height.
- Switching from 10-minute data to hourly data changes all the calculated parameters and changes the errors of the parameters at 200 m for both laws.

In light of these conclusions, it was decided to use hourly data for the purposes of comparison from this point on. The Weibull curve plotted for the Bremen station using the hourly data and the Distribution Fitter tool is given in the appendix on figure A.9. The parameters obtained from this curve are given alongside the correct reference values on figure 8.

It should be mentioned that a change was implemented during these steps for the roughness length value of water bodies. In the DWD reports, this type of terrain corresponds to 0.0 m roughness length [22]. However, it is not possible to implement the mathematical models using this roughness length, since it does not yield proper results. To solve this issue, research was conducted to find an alternative value. It was found that the value 0.0002 m is an acceptable roughness length value for ‘water surfaces: seas and lakes’ [20]. Therefore, this value was substituted for the 0 m roughness length value for the calculations.

With the changed parameters and errors, evaluation of the performance of the two laws was necessary. It was observed that the Modified Power Law performed better for Bremen and Rostock, whereas it performed much worse for Schmücke. Following these observations, it was decided to select more stations with similar roughness lengths to confirm how the laws perform for different terrains. Therefore, the weather stations Kahler Asten (station code 02483) and Hannover (station code 02014) were chosen to be investigated [22]. The weighted mean roughness lengths of the

Kahler Asten and Hannover stations were calculated to be 0.26 m and 0.11 m, respectively [22]. Another weather station with a low roughness length such as Rostock was not considered since the project supervisor advised that this type of terrain is not a significantly relevant one for the purpose of this project.

	Weibull A (m/s)		Weibull K		Mean Speed (m/s)	
Height (m)	Reference	Measured	Reference	Measured	Reference	Measured
10	4.8	4.81	1.87	1.95	4.22	4.26
	Weibull A (m/s)		Weibull k		Mean Speed (m/s)	
Height (m)	Reference	Power Law	Log Law	Reference	Power Law	Log Law
200	9.2	10.09	7.92	2.38	2.73	1.95
Error at 200m		0.89	1.28		0.35	0.43

Figure 8: Parameters obtained from the Weibull curve of Bremen

Combination of the two Laws

When the two laws were applied to the two new weather stations, some previous observations about the performance of the laws were confirmed as similar results were obtained [34]. The performance of the two laws for the 5 stations considered up to this point are summarized on table 13 [22].

Station and Roughness Length	Error at 200 m					
	Weibull A (m/s)		Weibull k		$\bar{v}$ (m/s)	
	Modified Power Law	Logarithmic Law	Modified Power Law	Logarithmic Law	Modified Power Law	Logarithmic Law
Bremen, 0.101 m	+0.89	-1.28	+0.35	-0.43	+0.828	-1.14
Rostock, 0.0304 m	-2.69	-2.97	+0.046	-0.28	-2.39	-2.63
Schmücke, 0.3185 m	+1.939	+0.81	+0.29	-0.22	+1.7	+0.7
Kahler Asten, 0.26 m	+1.444	+0.734	+0.38	-0.20	+1.34	+0.62
Hannover, 0.11 m	+0.93	-1.38	+0.49	-0.28	+0.85	-1.22

Table 13: Performance of the two Laws at different Weather Stations

From these error values, the following conclusions about the performance of the two mathematical wind profile models were made:

- The Modified Power Law performs better for roughness lengths around 0.1 m and for a very low roughness length.
- The Logarithmic Law performs better for roughness lengths above 0.2 m.
- The Modified Power Law overestimates the parameters increasingly with increasing roughness length.
- The Logarithmic Law underestimates the parameters for roughness lengths around 0.1 m, and starts overestimating increasingly with increasing roughness length at a value below 0.26 m.
- Both laws significantly underestimate the parameters for a very low roughness length.

These performance patterns show that both laws have their strengths and weaknesses. Therefore, it was decided that a combination of the two laws would be attempted as the final step of the project. To do this,

the wind speed values calculated at 200 m height by the two laws were investigated on the Excel spreadsheets. The following steps were followed:

- To have a reference in terms of the accuracy of estimation, the ratios between the parameters at the measured height and 200 m on the DWD report were checked [22].
- The same ratios were calculated for the measured and calculated data on the Excel spreadsheets.
- The DWD ratios were compared with the ones calculated on Excel to observe the different situations where a law over/undershoots the reference ratio, and to what extent [22].
- Additionally, the over/underestimation of the parameters by the two laws were used as reference in order to find situations in which one law performs better than the other, although performing worse in the larger point of view of the three parameters.

Following these steps, several situations in which different laws perform well were found. These situations depend mainly on the different roughness lengths, and in some cases on the measured wind speeds. These can be written as:

- The Logarithmic Law performs better for the roughness lengths 0.001 m and 0.0002 m.
- Between 0.001 m and 0.09 m roughness lengths, there are situations in which either of the laws perform better. However, the Modified Power Law performs better for the majority of these datapoints.
- If the measured wind speed is below 3 m/s, the Logarithmic Law performs better. However, this is only valid for the roughness lengths between around 0.1 m and 0.2 m.
- For roughness lengths above 0.2 m, the Logarithmic Law performs better with only one exception, which is the situation in which the measured wind speed is above 10 m/s.

According to these situations, some numerical ‘conditions’ for using the two laws were devised, attempting to cover each kind of roughness length. These are given on table 14.

Conditions	Roughness Length, z_0 (m)	$z_0 \leq 0.001$	$0.001 < z_0 \leq 0.09$	$0.09 < z_0 < 0.2$		$z_0 \geq 0.2$	
	Measured Wind Speed (m/s)	No condition	No condition	≤ 3	> 3	≤ 10	> 10
Applied Law	Modified Power Law		✓		✓		✓
	Logarithmic Law	✓		✓		✓	

Table 14: Different conditions for the usage of the two mathematical wind profile models

These conditions were applied to the calculations on the Excel spreadsheets for each station. The obtained parameters and errors at 200 m height for the Rostock, Schmücke, Kahler-Asten and Hannover weather stations are given on figures A.10-A.13, respectively. The results of the Bremen station are given in figure 9 as a reference for the conclusions that will be mentioned below. The results show that:

- The combination of the two laws according to these conditions results in quite low error values at 200 meters for roughness lengths close to 0.1 m, which is a rather relevant type of terrain for the project.
- For very low roughness lengths, the combined laws result in a lower error value but still underestimate the parameters.
- For higher roughness lengths, mainly the Logarithmic Law is used, but the incorporation of the Modified Power Law for certain datapoints lowers the error.

It is obvious that the combination of the two laws improves upon the usage of the Logarithmic Law alone for every case, which had been done by the previous project groups.

	Weibull A (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	9.2	9.55	10.09	7.92
Error at 200m		0.35	0.89	1.28
	Weibull k			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	2.38	2.25	2.73	1.95
Error at 200m		0.13	0.35	0.43
	Mean Speed (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	8.15	8.47	8.978	7.01
Error at 200m		0.32	0.828	1.14

Figure 9: Parameters and errors of the combined laws for Bremen

Programming

The stations are critical to understanding this project; they are constructed in different locations in several states in Germany, and their main purpose is to measure and store weather data periodically.

zehn_min_ff_Beschreibung_Stationen:

Source: https://opendata.dwd.de/climate_environment/CDC/help/zehn_min_ff_Beschreibung_Stationen.txt

This file is saved in “ProjectWork/Wind_Codes/local_data_and_classes” and contains information on the stations:

Stations_id	von_datum	bis_datum	Stationshoehe	goeBreite	goeLange	Stationsname	BundesLand	Abgabe
00003	19930429	20110331	202	50.7827	6.0941	Aachen	Nordrhein-Westfalen	Frei
00011	19920918	20241104	680	47.9736	8.5205	Donauwieschingen(Landeplatz)	Baden-Württemberg	Frei
00090	19910101	20241103	305	50.7557	9.2583	Alsfeld	Hessen	Frei
00096	20190410	20241104	50	52.9437	12.8518	Neuruppin-Alt-Ruppin	Brandenburg	Frei
00161	20110901	20241104	75	50.4237	7.4202	Andernach	Rheinland-Pfalz	Frei
00164	19911102	20241104	54	53.0316	13.9908	Angermünde	Brandenburg	Frei
00183	19911102	20241104	42	54.6791	13.4344	Arkona	Mecklenburg-Vorpommern	Frei
00197	19990421	20241104	365	51.3219	9.0558	Arnolds-Volkhardinghausen, Bad	Hessen	Frei
00198	19911108	20241104	164	51.3744	11.2920	Artern	Thüringen	Frei
00222	19911102	20080526	393	50.5908	12.7139	Aue	Sachsen	Frei
00232	19961108	20241104	462	48.4253	10.9417	Augsburg	Bayern	Frei
00282	19950601	20241104	240	49.8743	10.9206	Bamberg	Bayern	Frei
00298	19911101	20241104	3	54.3405	12.7108	Barth	Mecklenburg-Vorpommern	Frei
00303	19930907	20241104	55	52.0613	13.4997	Baruth	Brandenburg	Frei
00342	20101202	20241104	103	52.3170	8.1694	Bele	Niedersachsen	Frei
00348	19910102	20110901	127	50.4135	7.5886	Bendorf	Rheinland-Pfalz	Frei
00399	19911102	20150630	36	52.5198	13.4057	Berlin-Alexanderplatz	Berlin	Frei
00427	19911102	20241104	46	52.3807	13.5306	Berlin-Brandenburg	Brandenburg	Frei
00430	19941223	20210505	36	52.5644	13.3088	Berlin-Tegel	Berlin	Frei
00433	19950601	20241104	48	52.4676	13.4020	Berlin-Tempelhof	Berlin	Frei
00460	19931001	20241104	362	49.2641	6.6868	Berug	Saarland	Frei
00554	19940526	20060225	23	51.8293	6.5365	Bocholt-Liedern(Wasserwerk)	Nordrhein-Westfalen	Frei
00591	19911101	20241104	45	53.3911	10.6878	Boizenburg	Mecklenburg-Vorpommern	Frei
00596	19911107	20241104	15	54.0027	11.1908	Boltenhagen	Mecklenburg-Vorpommern	Frei
00603	20071031	20241104	147	50.7293	7.2040	Königswinter-Heiderhof	Nordrhein-Westfalen	Frei
00619	19980117	20241103	12	53.5788	6.6703	Borkum-Süderstraße	Niedersachsen	Frei
00642	19951124	20241104	4	53.3518	8.4969	Brake	Niedersachsen	Frei
00656	19910815	20241104	607	51.7234	10.6021	Braunlage	Niedersachsen	Frei
00662	19970702	20241104	81	52.2915	10.4464	Braunschweig	Niedersachsen	Frei
00691	19910620	20241104	4	53.0451	8.7981	Bremen	Bremen	Frei
00701	19971120	20241104	7	53.5332	8.5761	Bremerhaven	Bremen	Frei
00704	20020717	20241104	11	53.4451	9.1390	Bremervörde	Niedersachsen	Frei

[**Station_id** - Station number, **von_datum** - historical start date, **bis_datum** - historical start date, **Stationshoehe** – station height, **goeBreite** – latitude, **goeLange** – longitude, **Stationsname** – station name, **Bundesland** – State]

How to use or improve it

Download a more recent file if needed	Replace the spaces present within the station names or states with (- or ,)
Some station names or state names may contain “?” instead of (äöü), fix that	If a station name has two names separated by ‘,’ you may put the name after the comma in a bracket ()

weatherstation.py

This file contains a class called ‘Station’ whose purpose is to treat the stations from zehn_min_ff_Beschreibung_Stationen as objects, the class attributes given below:

The first 8 attributes are given names that correspond to the information/columns of the zehn_min_ff_Beschreibung_Stationen i.e. self.id, self.start_date, self.end_date, self.height, self.latitude, self.longitude, self.name, self.state. The rest are:

Class Attribute	Focus Class Method(s)	Description
self.user_chose_list	self.search()	This attribute is a Boolean that tells if the user typed in 'list' at the first input prompt. This allows the user to search for several stations at the same time using their station IDs. The result will be stored in self.list_of_stations
self.list_of_stations	self.search(), self.get_stations()	This is a list a list that contains the several stations that were searched at the same time.
self.roughness_lengths	self.fetch_roughness_lengths()	This attribute is a dictionary of the anemometer height and roughness lengths of the station
self.should_print	self.match(), self.info()	This attribute is a Boolean that determines if the information of the station(s) should be printed after a successful search. Default Value is False
self.user_pick	self.match()	The attribute is a Boolean that determines if the user should pick one out of the several stations in a chosen state. Default Value is False
self.user_chose_a_state	self.match(), self.pick_station()	This attribute is a Boolean with a default value of False, which changes to True when the user types in state name, at the first Input Prompt.

roughness_length_retrieve.py

This code gets the different anemometer heights and roughness lengths of the individual stations needed for wind calculations from the source below using **selenium**(web automation):

https://arla-sentinel.th-deg.de:8443/uploads/DWD_Winddaten_Version6/Frameseiten/Navigationsseite.html

Example: Anemometer height of station Aachen

Aachen: Dokumentation zur Auswertung der Messung

Der Bericht wurde mit dem Programm WAsP OWC Wizard (Version 2.0.62) erstellt und dokumentiert die Auswertung der Windmessung am Standort.

Zeitraum der ausgewerteten Daten: 01.01.1990 bis 31.12.2004; Koordinaten des Standortes: 50,78°N 6,10°E; Anemometerhöhe über Grund: 16 m (effektiv: 13 m).

Roughness lengths of Aachen

Informationen zu den modellierten Umgebungseinflüssen

Tabelle 2: Parameter der strömungstechnisch wirksamen Umgebungsbedingungen für die 12 Richtu

Grad [°]	Rau.-Wechsel	Referenz Rau.	FF-Korrektu
0	4	0,062	
30	4	0,080	
60	2	0,142	
90	1	0,179	
120	1	0,182	
150	2	0,173	
180	1	0,209	
210	1	0,226	
240	1	0,185	
270	2	0,141	
300	3	0,109	
330	3	0,104	

This code gets information from all the stations using their unique Xpath links on the website; this is implemented with a Python dictionary and several For loops

The result is stored in “ProjectWork/Wind_Codes/retrieved_data/roughness_lengths.txt”

How to use or improve the code:

- Ensure you have installed the necessary modules on your Python interpreter (time, os, selenium, from selenium.webdriver.common.by import By)
- Ensure you have Google Chrome with the correct Google Chrome Webdriver installed (depends on your Google Chrome version)
- If you want to retrieve something different you can edit within lines 47-71, otherwise, **do not touch the code**. The print function is your friend, in editing and debugging the code.

Wind Retrieval

This section discusses the retrieval of wind speeds from

https://opendata.dwd.de/climate_environment/CDC/observations_germany/climate/10_minutes/wind/historical/

and calculations of power in kW and energy in kWh.

Methodology:

Two Python classes (Station in weatherstation.py and DateFetcher in dates_fetcher.py) are used in conjunction with the wind_formulas.py file in order to achieve the goal. The aforementioned Python files are imported as modules in the wind_retrieve.py file along with other libraries.

```
8 import httpx
9 import asyncio
10 import pandas as pd
11 import zipfile
12 import io
13 import os
14 from local_data_and_classes.dates_fetcher import DatesFetcher
15 from selenium import webdriver
16 from selenium.common import NoSuchElementException
17 from selenium.webdriver.common.by import By
18 from formulas.wind_formulas import *
19 from local_data_and_classes.weatherstation import Station
20
```

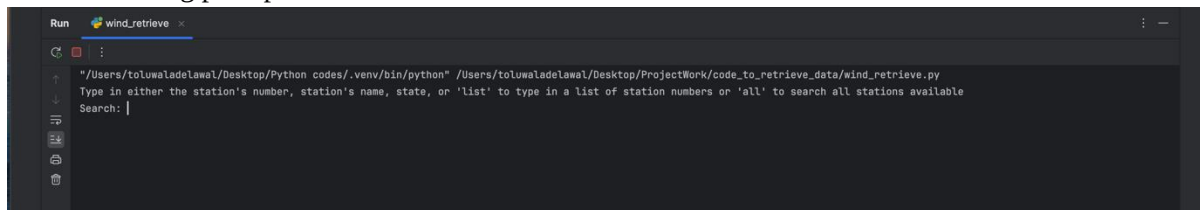
wind_retrieve.py

When the code is run, the retrieve_data() function is called, which starts by initializing the necessary variables

{‘total_data’: which stores all the retrieved data from the dwd website, ‘station_list_dates’: this stores the start and end dates of each final station used,

‘final_stations_used’: the name is self-explanatory}

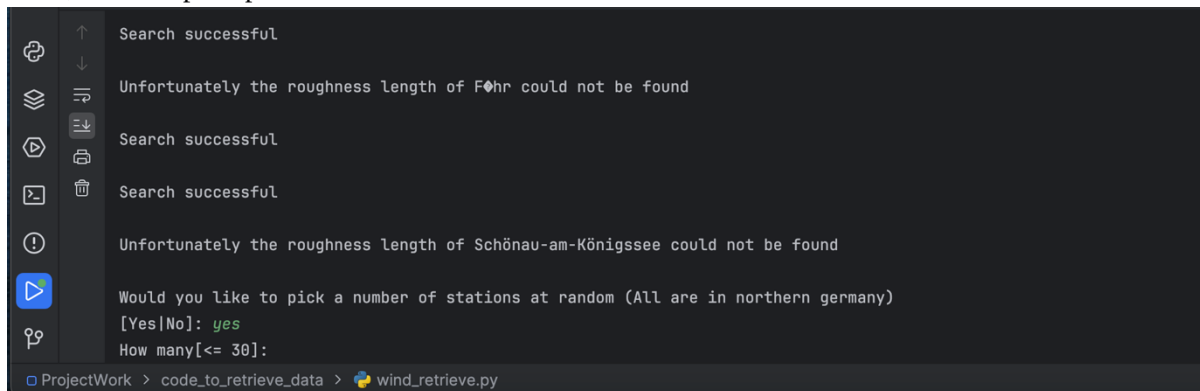
A station object is then created as 'stations', and the class method 'stations.search()' is then called resulting in the following prompt:



```
Run wind_retrieve x
"/Users/toluwaladelawal/Desktop/Python codes/.venv/bin/python" /Users/toluwaladelawal/Desktop/ProjectWork/code_to_retrieve_data/wind_retrieve.py
Type in either the station's number, station's name, state, or 'list' to type in a list of station numbers or 'all' to search all stations available
Search: |
```

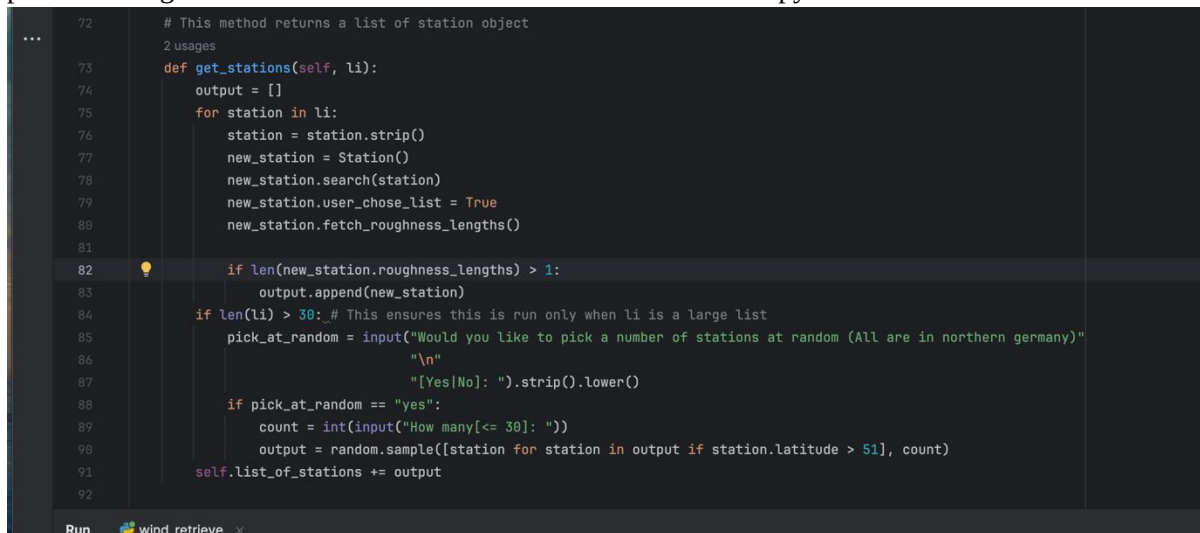
“all” is typed into the prompt, resulting in the search for all stations in the zehn_min_ff_Beschreibung_Stationen.txt file, as well as their roughness lengths (removing the stations whose roughness lengths could not be found and scraped from the arla_sentinel website).

Then, the next prompt is seen:



```
Search successful
Unfortunately the roughness length of Föhr could not be found
Search successful
Search successful
Unfortunately the roughness length of Schönaun-am-Königssee could not be found
Would you like to pick a number of stations at random (All are in northern germany)
[Yes|No]: yes
How many[<= 30]:
```

Due to the long run time associated with selenium and the extraction of wind speed data from over 200 stations on the website, the number of stations was limited to around 30 present in northern Germany (latitude > 51) (because of greater windspeeds in the north), chosen at random. This was made possible using a method of the Station class in the weatherstation.py file:



```
72 # This method returns a list of station object
73 2 usages
74 def get_stations(self, li):
75     output = []
76     for station in li:
77         station = station.strip()
78         new_station = Station()
79         new_station.search(station)
80         new_station.user_chose_list = True
81         new_station.fetch_roughness_lengths()
82     if len(new_station.roughness_lengths) > 1:
83         output.append(new_station)
84     if len(li) > 30: # This ensures this is run only when li is a large list
85         pick_at_random = input("Would you like to pick a number of stations at random (All are in northern germany)"
86                                "\n"
87                                "[Yes|No]: ").strip().lower()
88         if pick_at_random == "yes":
89             count = int(input("How many[<= 30]: "))
90             output = random.sample([station for station in output if station.latitude > 51], count)
91     self.list_of_stations += output
92
```

Note: Several stations available on the website have up to 4 zip files containing historical windspeed data, in order to speed up the data extraction of such files after data retrieval using selenium, two libraries were used (htpx and asyncio) in wind_retrieve.py, allowing asynchronous requests.

```

155         for station in stations:
156             try:
157                 total_rows_removed = 0
158                 data = []
159
160                 cur_zip_files = zip_files.get(station.id)
161
162                 if len(cur_zip_files) == 0:
163                     raise NoSuchElementException
164
165                 task = [fetch(url) for url in cur_zip_files]
166                 responses = await asyncio.gather(*task)

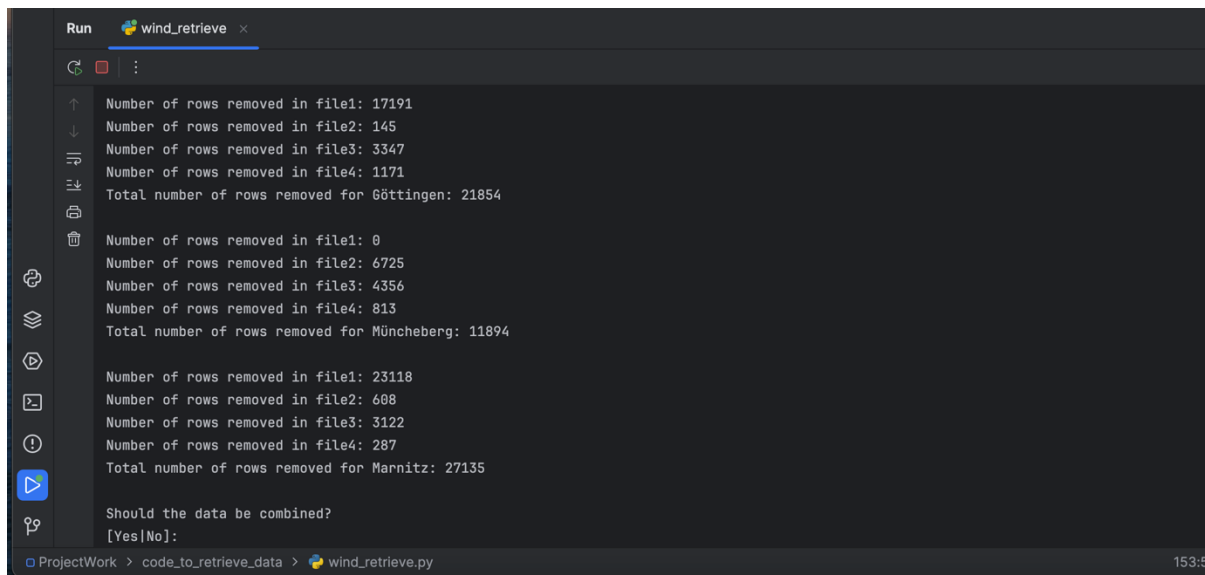
```

```

26
27 # This function allows several http requests at the same time
28 1 usage
29 async def fetch(url):
30     async with httpx.AsyncClient() as client:
31         response = await client.get(url)
32         return response

```

During the data extraction, rows containing erroneous data (-999) are removed for every file, the progress of this is printed in the run terminal, once everything is completed, the code asks if the data should be combined:



```

Run wind_retrieve x
Number of rows removed in file1: 17191
Number of rows removed in file2: 145
Number of rows removed in file3: 3347
Number of rows removed in file4: 1171
Total number of rows removed for Göttingen: 21854

Number of rows removed in file1: 0
Number of rows removed in file2: 6725
Number of rows removed in file3: 4356
Number of rows removed in file4: 813
Total number of rows removed for Müncheberg: 11894

Number of rows removed in file1: 23118
Number of rows removed in file2: 608
Number of rows removed in file3: 3122
Number of rows removed in file4: 287
Total number of rows removed for Marnitz: 27135

Should the data be combined?
[Yes/No]:

```

The next thing that appears in the prompt is intelligent code that compares the dates of the dates available for each station and guides the user in picking a date range. These are all made possible by the the DateFecther class in dates_fetcher.py

```

Select a date range. The dates available include the range
Start Date: 2006-11-15 15:40:00
End Date: 2021-05-04 23:50:00

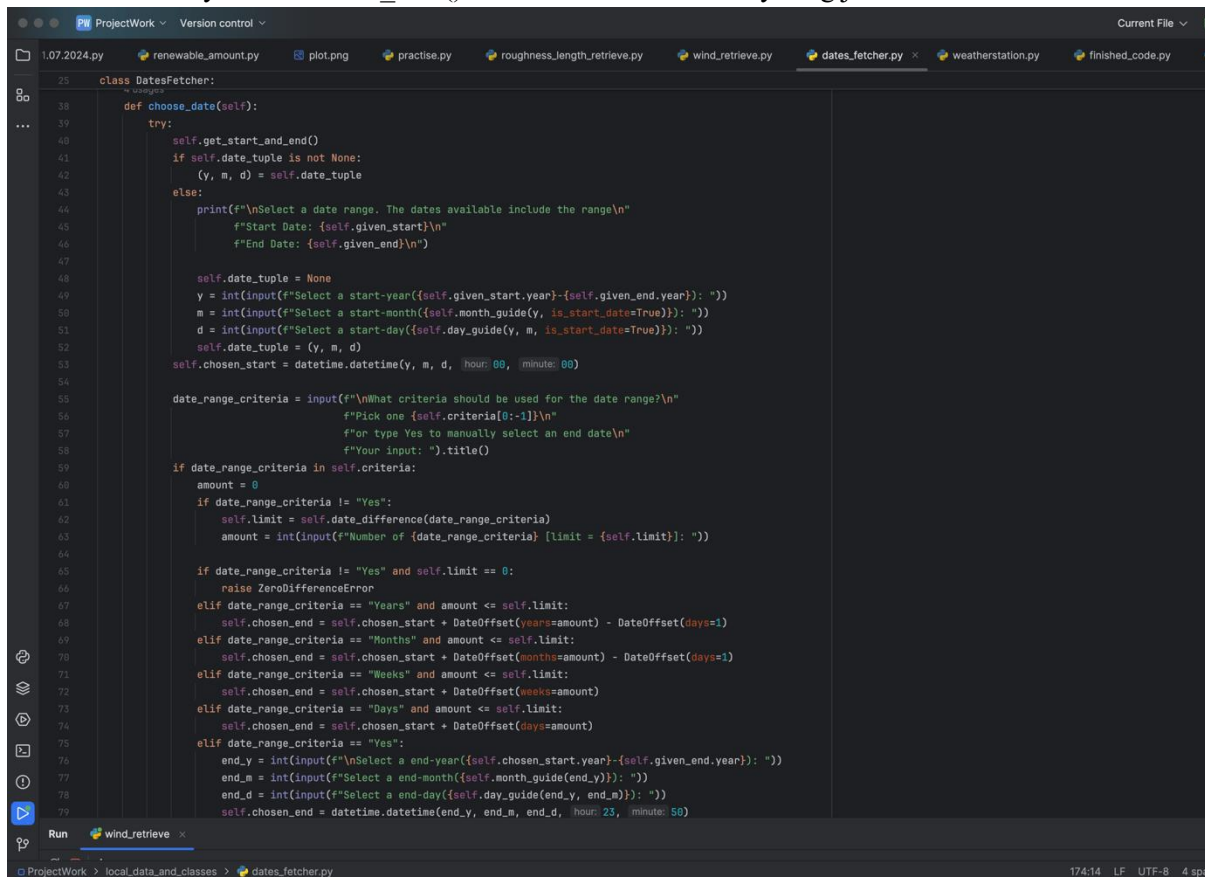
Select a start-year(2006-2021): 2021
Select a start-month(1-12): 1
Select a start-day(1-31): 1

What criteria should be used for the date range?
Pick one ['Days', 'Weeks', 'Months', 'Years']
or type Yes to manually select an end date
Your input: days
Number of Days [limit = 123]: 6

```

dates_fetcher.py

This file contains a class that deals with dates. It lets the user manually pick a start date, then select a date range using either days, weeks, months, or years away from the start date, the end date could also be selected manually. The “choose_date()” methods embodies everything just mentioned.

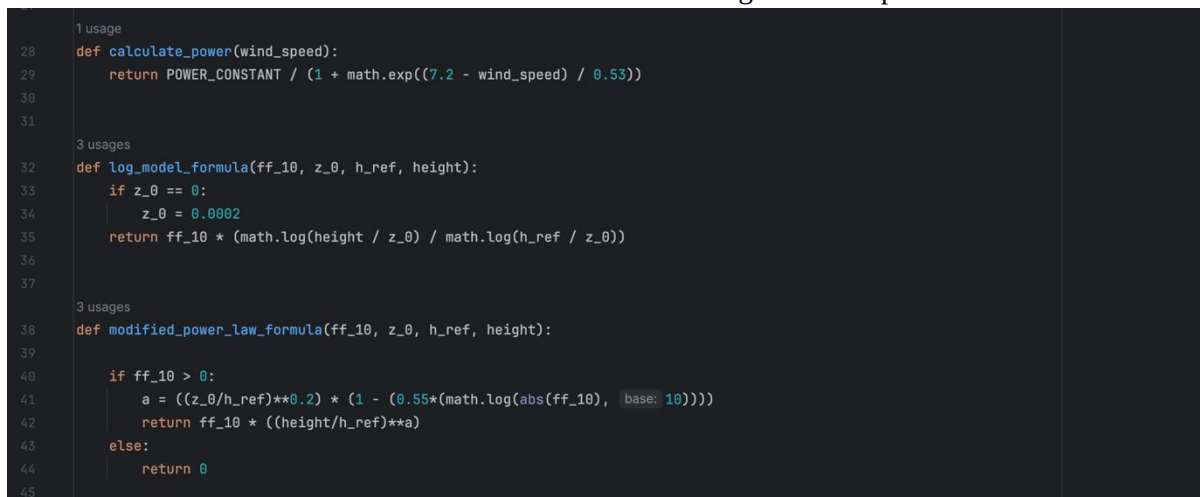


```
23 class DatesFetcher:
24     ...
25     def choose_date(self):
26         try:
27             self.get_start_and_end()
28             if self.date_tuple is not None:
29                 (y, m, d) = self.date_tuple
30             else:
31                 print(f"\nSelect a date range. The dates available include the range\n"
32                       f"Start Date: {self.given_start}\n"
33                       f"End Date: {self.given_end}\n")
34
35                 self.date_tuple = None
36                 y = int(input(f"Select a start-year({self.given_start.year}-{self.given_end.year}): "))
37                 m = int(input(f"Select a start-month({self.month_guide(y, is_start_date=True)}): "))
38                 d = int(input(f"Select a start-day({self.day_guide(y, m, is_start_date=True)}): "))
39                 self.date_tuple = (y, m, d)
40                 self.chosen_start = datetime.datetime(y, m, d, hour=00, minute=00)
41
42                 date_range_criteria = input(f"\nWhat criteria should be used for the date range?\n"
43                                           f"Pick one {self.criteria[0:-1]}\n"
44                                           f"or type Yes to manually select an end date\n"
45                                           f"Your input: ").title()
46
47                 if date_range_criteria in self.criteria:
48                     amount = 0
49                     if date_range_criteria != "Yes":
50                         self.limit = self.date_difference(date_range_criteria)
51                         amount = int(input(f"Number of {date_range_criteria} [limit = {self.limit}]: "))
52
53                     if date_range_criteria != "Yes" and self.limit == 0:
54                         raise ZeroDifferenceError
55                     elif date_range_criteria == "Years" and amount <= self.limit:
56                         self.chosen_end = self.chosen_start + DateOffset(years=amount) - DateOffset(days=1)
57                     elif date_range_criteria == "Months" and amount <= self.limit:
58                         self.chosen_end = self.chosen_start + DateOffset(months=amount) - DateOffset(days=1)
59                     elif date_range_criteria == "Weeks" and amount <= self.limit:
60                         self.chosen_end = self.chosen_start + DateOffset(weeks=amount)
61                     elif date_range_criteria == "Days" and amount <= self.limit:
62                         self.chosen_end = self.chosen_start + DateOffset(days=amount)
63                     elif date_range_criteria == "Yes":
64                         end_y = int(input(f"\nSelect a end-year({self.chosen_start.year}-{self.given_end.year}): "))
65                         end_m = int(input(f"Select a end-month({self.month_guide(end_y)}): "))
66                         end_d = int(input(f"Select a end-day({self.day_guide(end_y, end_m)}): "))
67                         self.chosen_end = datetime.datetime(end_y, end_m, end_d, hour=23, minute=50)
68         except:
69             pass
```

All other methods in this class help with its functionality. This method returns the start and end date chosen by the user.

wind formulas.py

This is the file that contains the different models for calculating the wind speed



```
1 usage
28 def calculate_power(wind_speed):
29     return POWER_CONSTANT / (1 + math.exp((7.2 - wind_speed) / 0.53))
30
31 3 usages
32 def log_model_formula(ff_10, z_0, h_ref, height):
33     if z_0 == 0:
34         z_0 = 0.0002
35     return ff_10 * (math.log(height / z_0) / math.log(h_ref / z_0))
36
37 3 usages
38 def modified_power_law_formula(ff_10, z_0, h_ref, height):
39
40     if ff_10 > 0:
41         a = ((z_0/h_ref)**0.2) * (1 - (0.55*(math.log(abs(ff_10), base=10))))
42         return ff_10 * ((height/h_ref)**a)
43     else:
44         return 0
45
```

And the code that decides which model should be used

```

47 # This calculates the wind speed at a new height using math models
48 1 usage
49 def calculate_vz(ff_10, z_0, h_ref, height):
50     """
51     :param ff_10: Speed of wind at the reference height, (Vref)
52     :param z_0: Roughness length
53     :param h_ref: Reference height, (Anemometer height)
54     :param height: Height at which the speed should be calculated
55     """
56     # This is the condition that determines which model is to be used
57     if z_0 <= 0.001:
58         wind_speed = log_model_formula(ff_10, z_0, h_ref, height)
59     elif 0.001 < z_0 <= 0.09:
60         wind_speed = modified_power_law_formula(ff_10, z_0, h_ref, height)
61     elif 0.09 < z_0 < 0.2:
62         if ff_10 <= 3:
63             wind_speed = log_model_formula(ff_10, z_0, h_ref, height)
64         else:
65             wind_speed = modified_power_law_formula(ff_10, z_0, h_ref, height)
66     else: # z_0 >= 0.2
67         if ff_10 <= 10:
68             wind_speed = log_model_formula(ff_10, z_0, h_ref, height)
69         else:
70             wind_speed = modified_power_law_formula(ff_10, z_0, h_ref, height)
71     return wind_speed
72
73
74
75

```

wind_retrieve.py results

There are two possible types of outputs; an excel file of an individual station:

	A	B	C	D	E	F	G	H	I	J	K
1	STATIONS_ID	MESS_DATUM	QN	FF_10	DD_10	eor	z0	V(200)	Power	Energy	
2	3366	2023-01-01 00:00:00	3	0.5	20	eor	0.098	0.8238368	3.874E-05	6.456E-06	
3	3366	2023-01-01 00:10:00	3	0.5	360	eor	0.102	0.8266622	3.894E-05	6.491E-06	
4	3366	2023-01-01 00:20:00	3	0.6	330	eor	0.091	0.9824762	5.225E-05	8.709E-06	
5	3366	2023-01-01 00:30:00	3	0.4	200	eor	0.077	1.5893464	0.0001642	2.737E-05	
6	3366	2023-01-01 00:40:00	3	0.8	110	eor	0.065	2.5328586	0.0009738	0.0001623	
7	3366	2023-01-01 00:50:00	3	1.3	90	eor	0.069	3.6703573	0.0083193	0.0013865	
8	3366	2023-01-01 01:00:00	3	1.5	70	eor	0.088	4.2861661	0.0265139	0.004419	
9	3366	2023-01-01 01:10:00	3	0.9	100	eor	0.069	2.8005789	0.0016137	0.0002689	

Or for several stations combined:

	A	B	C	D	E	F	G
1	MESS_DATUM	Power	Energy				
2	2022-01-01 00:00:00	144.42542	24.070903				
3	2022-01-01 00:10:00	143.15402	23.859003				
4	2022-01-01 00:20:00	147.09906	24.51651				
5	2022-01-01 00:30:00	147.55602	24.592669				
6	2022-01-01 00:40:00	147.14461	24.524102				

When the stations are combined, information of all the individual stations that were combined are stored in a text file sharing the same name as the excel file, for easy reference:

```
renewable_amount.py combinedwindnw_2022-01-01_to_2022-12-31.txt x zehn_min_ff_Beschreibung_Stationen.txt zehn_m
1 {'Stations': ['Dörpen', 'Ueckermünde', 'Kahler-Asten', 'Nossen', 'Brake', 'Lügde-Paenbruch', 'Münster/Osnabrück',
2 'Dörpen': {'Power': 131607.37306133154, 'Energy': 21934.562176888383}
3 'Ueckermünde': {'Power': 94945.38816701436, 'Energy': 15824.231361169423}
4 'Kahler-Asten': {'Power': 207897.82828709483, 'Energy': 34649.63804784944}
5 'Nossen': {'Power': 119815.80929110886, 'Energy': 19969.301548516494}
6 'Brake': {'Power': 171719.29800574313, 'Energy': 28619.883000955542}
7 'Lügde-Paenbruch': {'Power': 128008.48970980906, 'Energy': 21334.748284970534}
8 'Münster/Osnabrück': {'Power': 93602.88437357842, 'Energy': 15600.480728931496}
```

Solar Retrieval

In order to evaluate models that simulate diffuse solar radiation and their association with actual data from the German Weather Service (DWD), two Python scripts for processing and analyzing solar irradiance data are used. The Perez model and Global Horizontal Solar (GHS) radiation data from 2020–2023 are used in the scripts. While the second script visualizes models versus actual diffuse solar radiation and conducts a correlation analysis, the first script computes solar energy parameters by processing irradiance and meteorological data.

This code begins by importing the necessary libraries for data processing and visualization.

The **input files** contain solar irradiance and diffuse sunlight data for the period from 2020 to 2023 . Additional meteorological data, including solar zenith angle, azimuth, and air mass, which are crucial for determining solar radiation and angles

```
1
2 import numpy as np
3 import pandas as pd
4 import datetime as dt
5 import matplotlib.pyplot as plt
6
7 # Load input data
8 Datei = "produkt_zehn_min_sd_20200101_20231231_03366.txt"
9 panin = "angels_mühlendorf.txt"
10
11 # Read the file
12 dwd_data = pd.read_csv(Datei, sep=";")
13 zen_data = pd.read_csv(panin, delim_whitespace=True)
14
15 # Extract and preprocess data
16 solar_zenith = zen_data['SolarZen']
17 azimuth = zen_data['Azim']
18 air_mass = zen_data['AirMass']
19 diffuse_sunlight = dwd_data['DS_10']
20 global_sunlight = dwd_data['GS_10']
21
22 GHS = np.maximum("args: 0, (global_sunlight.astype(float) / 600) * 1e4 # Convert to W/m^2
23 DHI = np.maximum("args: 0, (diffuse_sunlight.astype(float) / 600) * 1e4 # Convert to W/m^2
24 DI = np.maximum("args: 0, GHS - DHI
```

In this section, the data is loaded from the files using `pandas.read_csv`. The **DWD data** (Datei) contains values for diffuse and global solar radiation (measured in W/m^2), while the **Zenith data** (panin) includes solar zenith angle, azimuth angle, and air mass. The following key data series are extracted:

```

122     # Hay and Davies Sky Diffuse Irradiance
123     DI_th[i] = DHI[i] * (A * R + (1 - A) * (1 + np.cos(np.radians(theta_t))) / 2)
124
125 # Output results
126 df = pd.DataFrame({
127     S4_    'Date': time_...,
128     'GHS': GHS_...,
129     'DHI': DHI_...,
130     'Theta_Z': theta_Z_...,
131     'DNI': DNI_...,
132     'Perez_Irradiance': DI_tp_...,
133     'Reindl_Irradiance': DI_tr_...,
134     'HayDavies_Irradiance': DI_th_...
135 })
136
137 # Save to Excel
138 df.to_excel(excel_writer, 'PerezModelOutput_Müldorf.xlsx', index=False)
139
140 # Plot results
141 plt.figure(figsize=(10, 12))
142 plt.subplot(*args: 4, 1, 1)
143 plt.plot(*args: time, DI_tp, 'r')
144 plt.title('Perez Diffuse Irradiance')
145
146 plt.subplot(*args: 4, 1, 2)
147 plt.plot(*args: time, DI_tr, 'g')
148 plt.title('Reindl Sky Diffuse Irradiance')
149
150 plt.subplot(*args: 4, 1, 3)
151 plt.plot(*args: time, DI_th, 'b')
152 plt.title('Hay and Davies Sky Diffuse Irradiance')
153
154 plt.subplot(*args: 4, 1, 4)
155 plt.plot(*args: time, DNI, 'k')
156 plt.title('Direct Normal Irradiance')
157
158 plt.tight_layout()
159 plt.show()
160

```

This code snippet ensures that any **invalid zenith angles** (greater than 87 degrees) are set to zero. It also prepares the azimuth angles for further calculations.

A time vector is created starting from January 1st, 2020, with a 10-minute interval. This forms the basis for the time-dependent nature of solar irradiance data. Empirical coefficients are predefined for various solar irradiance models, such as the **Perez model**. These coefficients are used to simulate **Diffuse Irradiance** using various weather parameters.

```

156 # Save to Excel
157 df.to_excel(excel_writer, 'PerezModelOutput_Müldorf.xlsx', index=False)
158
159 # Plot results
160 plt.figure(figsize=(10, 12))
161 plt.subplot(*args: 4, 1, 1)
162 plt.plot(*args: time, DI_tp, 'r')
163 plt.title('Perez Diffuse Irradiance')
164
165 plt.subplot(*args: 4, 1, 2)
166 plt.plot(*args: time, DI_tr, 'g')
167 plt.title('Reindl Sky Diffuse Irradiance')
168
169 plt.subplot(*args: 4, 1, 3)
170 plt.plot(*args: time, DI_th, 'b')
171 plt.title('Hay and Davies Sky Diffuse Irradiance')
172
173 plt.subplot(*args: 4, 1, 4)
174 plt.plot(*args: time, DNI, 'k')
175 plt.title('Direct Normal Irradiance')
176
177 plt.tight_layout()
178 plt.show()
179

```

This code imports additional libraries and reads the diffuse irradiance data from an Excel file containing model outputs for **Perez**, **Reindl**, and **Hay-Davies** models, as well as DHI values from the DWD. The `.head()` method is used to display the first five rows of the dataset to ensure that it is correctly loaded.

```
[2]
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

[3]
diffuse_models = pd.read_excel("PerezModelOutput_Hamburg.xlsx")

[4]
diffuse_models.head(5)
```

	Date	GHS	DHI	Theta_z	DNI	Perez_Irradiance	Reindl_Irradiance	HayDavies_Irradiance
0	2020-01-01 00:00:00	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	2020-01-01 00:10:00	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	2020-01-01 00:20:00	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	2020-01-01 00:30:00	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	2020-01-01 00:40:00	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Analysis of the Correlation Between Three Different Models and DHI Values from DWD

The heatmap analysis indicates that the three models exhibit a high degree of correlation with each other, as well as with the DHI values from DWD. The differences between the models are minimal, suggesting that they all perform similarly in terms of their relationship with the DHI data. This strong correlation describes the reliability and consistency across the models in predicting or aligning with the DHI values.

```
[5]
all_diffuse = diffuse_models[['DHI', 'Perez_Irradiance', 'Reindl_Irradiance', 'HayDavies_Irradiance']]

[17]
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

# Compute the correlation matrix for the encoded DataFrame
corr_matrix = all_diffuse.corr()

# Set the figure size (width, height)
plt.figure(figsize=(12, 8)) # Adjust the values (12, 8) as per your needs

Run roughness_length.retrieve x Python 3.11 (Python codes)
```

The code calculates the **correlation matrix** for the various solar irradiance models. This allows us to see how strongly each model correlates with the DHI values. A **heatmap** is generated to visually represent the correlation between different models and DHI. Strong correlations indicate that the models closely track DHI values.

```
[5]
all_diffuse = diffuse_models[['DHI', 'Perez_Irradiance', 'Reindl_Irradiance', 'HayDavies_Irradiance']]

[17]
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

# Compute the correlation matrix for the encoded DataFrame
corr_matrix = all_diffuse.corr()

# Set the figure size (width, height)
plt.figure(figsize=(12, 8)) # Adjust the values (12, 8) as per your needs

# Plot the heatmap
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.savefig('Model correlation_HB.png')

# Show the plot
plt.show()
```

In this section, the code creates a series of **subplots** to visualize the monthly irradiance data for the years 2020 through 2023.


```

# Extract the month and year from the 'Date' column
diffuse_models['Month'] = diffuse_models['Date'].dt.month
diffuse_models['Year'] = diffuse_models['Date'].dt.year

# Set up the plot with subplots
fig, axes = plt.subplots(2, 2, figsize=(15, 10), sharex=True)
fig.suptitle('Annual Comparison of Global Horizontal Irradiance (GHI) and Perez Irradiance in Hamburg', fontsize=16)

# Loop through each year and create subplots
for i, year in enumerate([2020, 2021, 2022, 2023]):
    ax = axes[i // 2, i % 2] # Determine subplot position
    year_data = diffuse_models[diffuse_models['Year'] == year]

    # Plot the GHI and Perez Irradiance for the year
    ax.plot(year_data['Month'], year_data['GHI'], label='GHI', color='orange', linewidth=2)
    ax.plot(year_data['Month'], year_data['Perez_Irradiance'], label='Perez Irradiance', color='green', linestyle='--', linewidth=2)

    # Plot the curve connecting the maximum values for each month (calculated for each year)
    monthly_max = year_data.groupby('Month').agg({
        'GHI': 'max',
        'Perez_Irradiance': 'max'
    }).reset_index()

    # Shaded area beneath GHI curve
    ax.fill_between(monthly_max['Month'], monthly_max['GHI'], color='orange', alpha=0.3)

    # Shaded area beneath Perez Irradiance curve
    ax.fill_between(monthly_max['Month'], monthly_max['Perez_Irradiance'], color='green', alpha=0.3)

    ax.plot(monthly_max['Month'], monthly_max['GHI'], color='orange', marker='o', linestyle='-', linewidth=2, label='Max GHI')
    ax.plot(monthly_max['Month'], monthly_max['Perez_Irradiance'], color='green', marker='x', linestyle='-', linewidth=2, label='Max Perez Irradiance')

    # Customize the plot
    ax.set_title(f'Year: {year}')
    ax.set_xlabel('Month')
    ax.set_ylabel('Value')
    ax.set_xticks(range(1, 13)) # Set X-axis labels for months
    ax.set_xticklabels(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
    ax.legend()

    # Set the Y-axis range from 0 to 1400 for both y-axes
    ax.set_ylim(0, 1400)

plt.savefig('Monthly Irradiance Analysis for 2020-2023_HB.png')

# Step 4: Show the plot
plt.tight_layout()
plt.show()

```

This section identifies the **months** with the highest and lowest **average Perez Irradiance** for each year. This helps in understanding seasonal trends in solar radiation.

```

[13]
# Convert 'Date' to datetime if not already
diffuse_models['Date'] = pd.to_datetime(diffuse_models['Date'])
diffuse_models['Year'] = diffuse_models['Date'].dt.year
diffuse_models['Month'] = diffuse_models['Date'].dt.month
diffuse_models['Day'] = diffuse_models['Date'].dt.day

# Data for maximum and minimum average Perez Irradiance
max_months = [
    (2020, 7),
    (2021, 7),
    (2022, 7),
    (2023, 5),
]
min_months = [
    (2020, 12),
    (2021, 12),
    (2022, 12),
    (2023, 12),
]

# Combine min and max months for sequential plotting
all_months = [(min_months[i], max_months[i]) for i in range(len(min_months))]

# Prepare the plot
fig, axes = plt.subplots(4, 2, figsize=(15, 20), sharex=True)
fig.suptitle('Daily Perez Irradiance for Minimum and Maximum Average Months', fontsize=16)

# Plotting each year's minimum and maximum data
for i, ((min_year, min_month), (max_year, max_month)) in enumerate(all_months):
    # Plot minimum for the year
    ax_min = axes[i, 0]
    min_data = diffuse_models[(diffuse_models['Year'] == min_year) & (diffuse_models['Month'] == min_month)]
    ax_min.bar(min_data['Day'], min_data['Perez_Irradiance'], color='red', label=f'Min Perez Irradiance ({min_month})')
    ax_min.set_title(f'Year: {min_year} - Min Month: {min_month}')
    ax_min.set_xlabel('Day of Month')
    ax_min.set_ylabel('Perez Irradiance')
    ax_min.legend()

    # Plot maximum for the year
    ax_max = axes[i, 1]
    max_data = diffuse_models[(diffuse_models['Year'] == max_year) & (diffuse_models['Month'] == max_month)]
    ax_max.bar(max_data['Day'], max_data['Perez_Irradiance'], color='blue', label=f'Max Perez Irradiance ({max_month})')
    ax_max.set_title(f'Year: {max_year} - Max Month: {max_month}')
    ax_max.set_xlabel('Day of Month')
    ax_max.set_ylabel('Perez Irradiance')
    ax_max.legend()

```

The final set of visualizations compares **daily Perez Irradiance** values for months with maximum and minimum irradiance. These insights are crucial for evaluating solar energy potential on a day-to-day basis in regions like Mühldorf.


```

fig.suptitle("Daily Perez Irradiance for Minimum and Maximum Average Months", 'fontsize16')

# Plotting each year's minimum and maximum data
for i, ((min_year, min_month), (max_year, max_month)) in enumerate(all_months):
    # Plot minimum for the year
    ax_min = axes[i, 0]
    min_data = diffuse_models[(diffuse_models['Year'] == min_year) & (diffuse_models['Month'] == min_month)]
    ax_min.bar(min_data['Day'], min_data['Perez_Irradiance'], color='red', label='Min Perez Irradiance ({min_month})')
    ax_min.set_title("{}Year (min_year) - Min Month (min_month)".format(min_year))
    ax_min.set_xlabel("Day of Month")
    ax_min.set_ylabel("Perez Irradiance")
    ax_min.legend()

    # Plot maximum for the year
    ax_max = axes[i, 1]
    max_data = diffuse_models[(diffuse_models['Year'] == max_year) & (diffuse_models['Month'] == max_month)]
    ax_max.bar(max_data['Day'], max_data['Perez_Irradiance'], color='blue', label='Max Perez Irradiance ({max_month})')
    ax_max.set_title("{}Year (max_year) - Max Month (max_month)".format(max_year))
    ax_max.set_xlabel("Day of Month")
    ax_max.set_ylabel("Perez Irradiance")
    ax_max.legend()

plt.savefig('max_min_barcharts.png')

# Adjust layout
plt.tight_layout(rect=[0, 0.05, 1, 0.95])
plt.show()

```

References

- [1] J. Mwaura and A. Krol, “Solar Energy,” *MIT Climate Portal*, Aug. 29, 2023. <https://climate.mit.edu/explainers/solar-energy> (accessed Dec. 19, 2024).
- [2] A. O. M. Maka and J. M. Alabid, “Solar energy technology and its roles in sustainable development,” *Clean Energy*, vol. 6, no. 3, pp. 476–483, Jun. 2022, doi: <https://doi.org/10.1093/ce/zkac023>.
- [3] “Definitions,” *SolarAnywhere*. <https://www.solaranywhere.com/support/data-fields/definitions/>
- [4] “Hay and Davies Sky Diffuse Model,” *PV Performance Modeling Collaborative (PVPMC)*, 2021. <https://pvpmc.sandia.gov/modeling-guide/1-weather-design-inputs/plane-of-array-poa-irradiance/calculating-poa-irradiance/poa-sky-diffuse/hay-and-davies-sky-diffuse-model/> (accessed Jan. 07, 2025).
- [5] P. G. Loutzenhiser, H. Manz, C. Felsmann, P. A. Strachan, T. Frank, and G. M. Maxwell, “Empirical validation of models to compute solar irradiance on inclined surfaces for building energy simulation,” *Solar Energy*, vol. 81, no. 2, pp. 254–267, Feb. 2007, doi: <https://doi.org/10.1016/j.solener.2006.03.009>.
- [6] D. T. Reindl, W. A. Beckman, and J. A. Duffie, “Diffuse fraction correlations,” *Solar Energy*, vol. 45, no. 1, pp. 1–7, 1990. [Online]. Available: [https://doi.org/10.1016/0038-092X\(90\)90060-P](https://doi.org/10.1016/0038-092X(90)90060-P)
- [7] Sandia National Laboratories, “Reindl Sky Diffuse Model – PV Performance Modeling Collaborative (PVPMC).” [Online]. Available: <https://pvpmc.sandia.gov/modeling-guide/1-weather-design-inputs/plane-of-array-poa-irradiance/calculating-poa-irradiance/poa-sky-diffuse/reindl-sky-diffuse-model/>
- [8] R. Mubarak, M. Hofmann, S. Riechelmann, and G. Seckmeyer, “Comparison of modelled and measured tilted solar irradiance for photovoltaic applications,” *Energies*, vol. 10, no. 11, p. 1688, 2017. [Online]. Available: <https://doi.org/10.3390/en10111688>
- [9] A. Driesse, A. R. Jensen, and R. Perez, “A continuous form of the Perez diffuse sky model for forward and reverse transposition,” *Solar Energy*, vol. 267, pp. 112093–112093, Dec. 2023, doi: <https://doi.org/10.1016/j.solener.2023.112093>.
- [10] R. Perez, R. Seals, P. Ineichen, R. Stewart, and D. Menicucci, “A new simplified version of the Perez diffuse irradiance model for tilted surfaces,” *Solar Energy*, vol. 39, no. 3, pp. 221–231, 1987, doi: [https://doi.org/10.1016/s0038-092x\(87\)80031-2](https://doi.org/10.1016/s0038-092x(87)80031-2).
- [11] “Angle of Incidence,” *PV Performance Modeling Collaborative (PVPMC)*. <https://pvpmc.sandia.gov/modeling-guide/1-weather-design-inputs/plane-of-array-poa-irradiance/calculating-poa-irradiance/angle-of-incidence/>
- [12] R. Perez, R. Stewart, C. Arbogast, R. Seals, and J. Scott, “An anisotropic hourly diffuse radiation model for sloping surfaces: Description, performance validation, site dependency evaluation,” *Solar Energy*, vol. 36, no. 6, pp. 481–497, 1986, doi: [https://doi.org/10.1016/0038-092x\(86\)90013-7](https://doi.org/10.1016/0038-092x(86)90013-7).
- [13] “Air Mass,” *PV Performance Modeling Collaborative (PVPMC)*. <https://pvpmc.sandia.gov/modeling-guide/1-weather-design-inputs/irradiance-insolation/air-mass/>
- [14] “1.5. Extraterrestrial Radiation and the Atmosphere | EME 811: Solar Thermal Energy for Utilities and Industry,” *Psu.edu*, 2023. <https://www.e-education.psu.edu/eme811/node/637>

- [15] R. Perez, P. Ineichen, R. Seals, J. Michalsky, and R. Stewart, “Modeling daylight availability and irradiance components from direct and global irradiance,” *Solar Energy*, vol. 44, no. 5, pp. 271–289, 1990, doi: [https://doi.org/10.1016/0038-092x\(90\)90055-h](https://doi.org/10.1016/0038-092x(90)90055-h).
- [16] “Perez Sky Diffuse Model,” PV Performance Modeling Collaborative (PVPMC), 2021. <https://pvpmc.sandia.gov/modeling-guide/1-weather-design-inputs/plane-of-array-poa-irradiance/calculating-poa-irradiance/poa-sky-diffuse/perez-sky-diffuse-model/>
- [17] R. Perez, R. Stewart, R. Seals, and T. Guertin, “The development and verification of the Perez diffuse radiation model,” *OSTI OAI (U.S. Department of Energy Office of Scientific and Technical Information)*, Oct. 1988, doi: <https://doi.org/10.2172/7024029>.
- [18] “V172-7.2 MWTM,” Global Leader in Sustainable Energy, <https://www.vestas.com/en/energy-solutions/onshore-wind-turbines/enventus-platform/V172-7-2-MW> (accessed Jan. 16, 2025).
- [19] “Log wind profile,” Wikipedia, https://en.wikipedia.org/wiki/Log_wind_profile (accessed Nov. 6, 2024).
- [20] christoph. schilter@meteotest. ch METEOTEST, “The Swiss Wind Power Data Website,” Windenergie-Daten der Schweiz, <https://wind-data.ch/tools/profile.php?lng=en> (accessed Oct. 30, 2024).
- [21] “Weibull distribution,” Wikipedia, https://en.wikipedia.org/wiki/Weibull_distribution (accessed Jan. 16, 2024).
- [22] Undisclosed Publication: Deutsche Wetterdienst Winddaten für Windenergienutzer. 2. Auflage, Version 6.
- [23] christoph. schilter@meteotest. ch METEOTEST, “Die website Für Windenergie-Daten der Schweiz,” Windenergie-Daten der Schweiz, <https://wind-data.ch/tools/weibull.php> (accessed Oct. 30, 2024).
- [24] “Estimate parameters of Weibull distribution,” MathWorks, <https://www.mathworks.com/help/stats/wblfit.html> (accessed Nov. 6, 2024).
- [25] “Wind profile power law,” Wikipedia, https://en.wikipedia.org/wiki/Wind_profile_power_law (accessed Nov. 14, 2024).
- [26] A. K. Resen, A. B. Khamees, and S. F. Yaseen, “Determination of wind shear coefficients and conditions of atmospheric stability for three Iraqi sites,” *IOP Conference Series: Materials Science and Engineering*, vol. 881, no. 1, p. 012161, Jul. 2020. doi:10.1088/1757-899x/881/1/012161
- [27] Weighted average calculator, <https://www.rapidtables.com/calc/math/weighted-average-calculator.html> (accessed Nov. 14, 2024).
- [28] D.A. Spera, T.R. Richards, 01.01.1979, “Modified power law equations for vertical wind profiles”. Presented at the Wind Characteristics and Wind Energy Siting Conference, 19-21 June 1979. Available: <https://ntrs.nasa.gov/api/citations/19800005367/downloads/19800005367.pdf>.

- [29] “Least square method - formula, definition, examples,” Cuemath, <https://www.cuemath.com/data/least-squares/> (accessed Nov. 18, 2024).
- [30] J. Frost, “Root mean square error (RMSE),” Statistics By Jim, <https://statisticsbyjim.com/regression/root-mean-square-error-rmse/> (accessed Nov. 18, 2024).
- [31] “Calculator Suite,” GeoGebra, <https://www.geogebra.org/calculator> (accessed Dec. 4, 2024).
- [32] “FitExp command,” FitExp Command :: GeoGebra Manual, <https://geogebra.github.io/docs/manual/en/commands/FitExp/> (accessed Dec. 4, 2024).
- [33] “Fitdist,” MathWorks, <https://de.mathworks.com/help/stats/distributionfitter-app.html> (accessed Dec. 11, 2024).
- [34] Deutsche Wetterdienst Climate Data Center, https://opendata.dwd.de/climate_environment/CDC/observations_germany/.
- [35] B. W. e.V., “Statistics germanybwe E.V.,” BWE e.V., <https://www.wind-energie.de/english/statistics/statistics-germany/> (accessed Jan. 22, 2025).
- [36] “Public electricity generation 2024: Renewable energies cover more than 60 percent of German electricity consumption for the first time - fraunhofer ise,” Fraunhofer Institute for Solar Energy Systems ISE, <https://www.ise.fraunhofer.de/en/press-media/press-releases/2025/public-electricity-generation-2024-renewable-energies-cover-more-than-60-percent-of-german-electricity-consumption-for-the-first-time.html> (accessed Jan. 22, 2025).
- [37] B. Wehrmann et al., “German onshore wind power – output, business and perspectives,” Clean Energy Wire, <https://www.cleanenergywire.org/factsheets/german-onshore-wind-power-output-business-and-perspectives> (accessed Jan. 22, 2025).

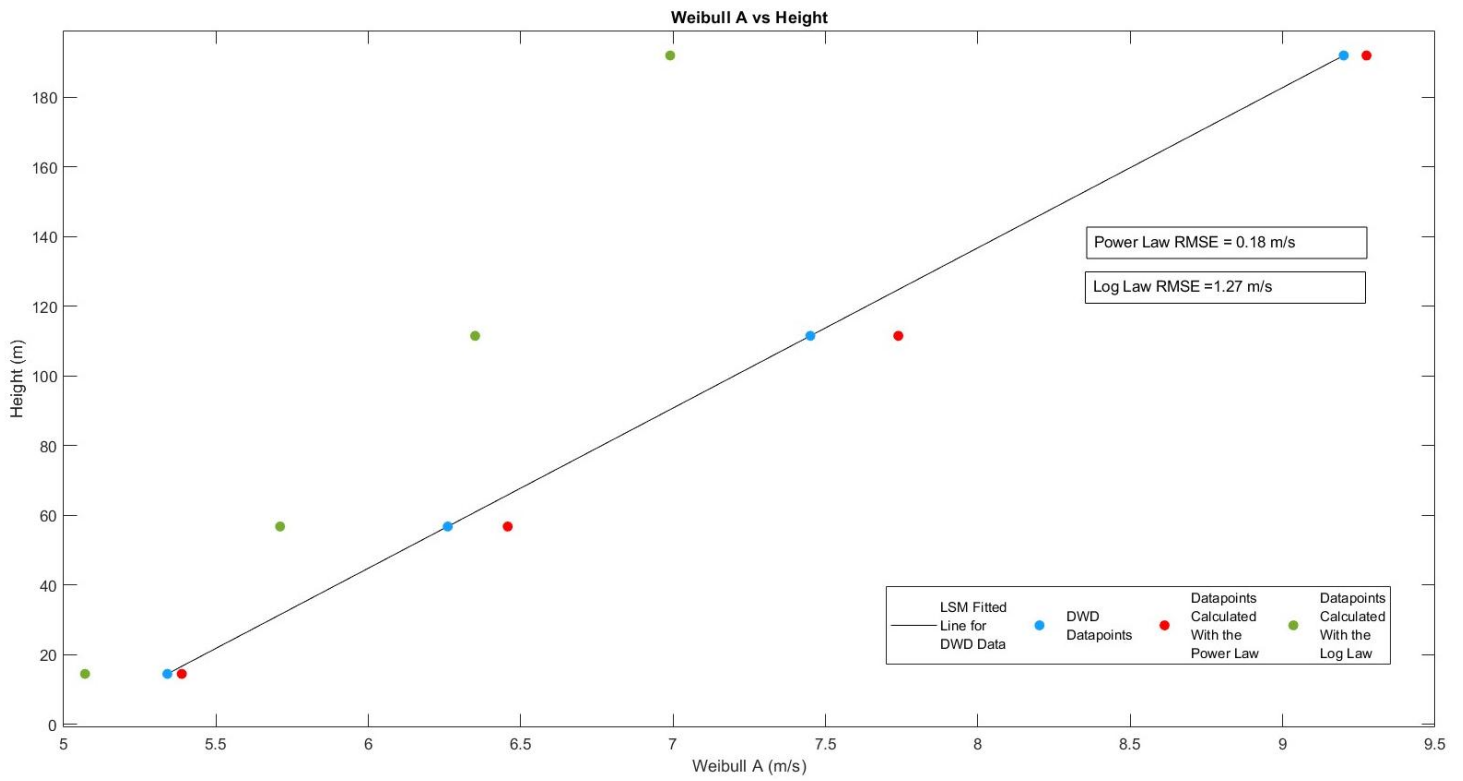


Figure A.1: Weibull A vs Height for Both Laws Bremen

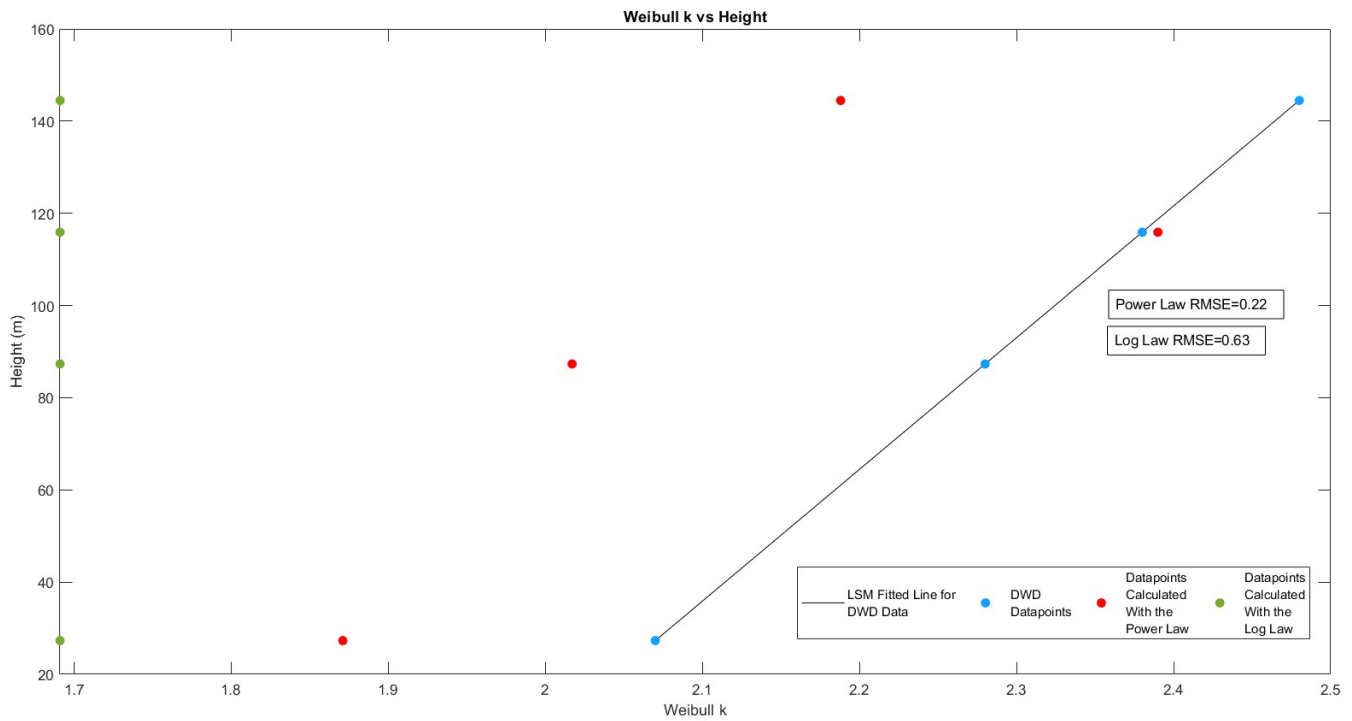


Figure A.2: Weibull k vs Height for Both Laws Bremen

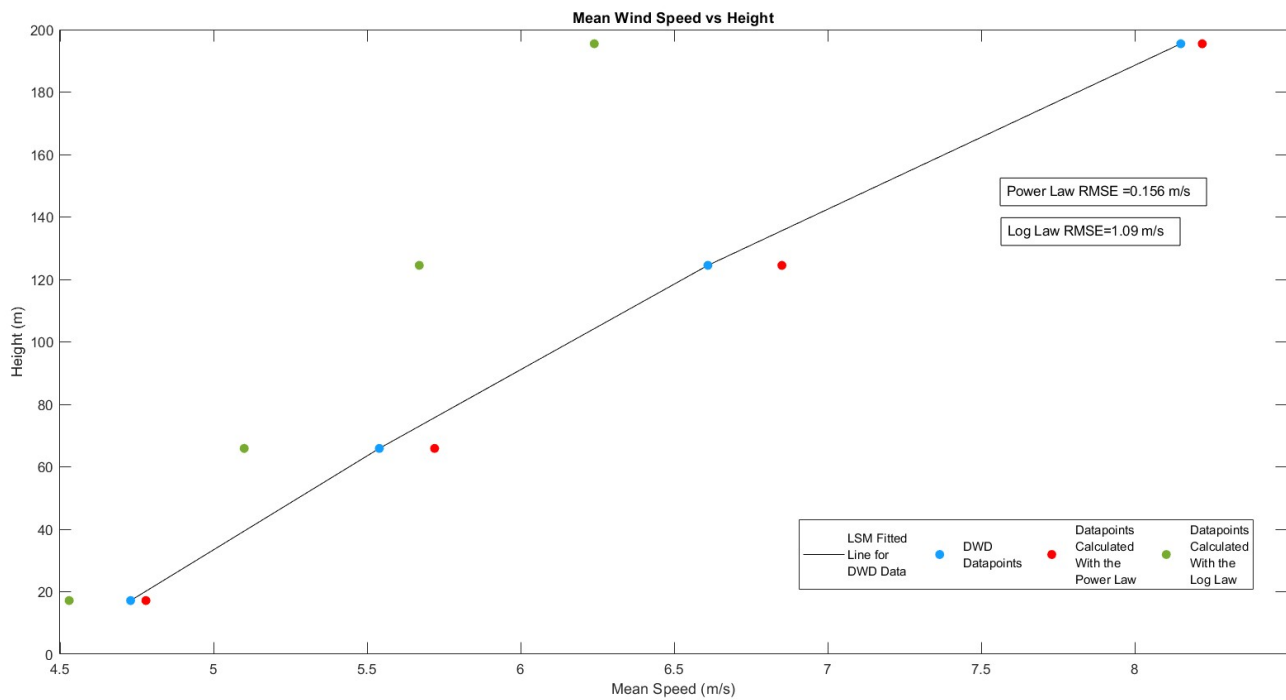


Figure A.3: Mean Speed vs Height for both Laws Bremen

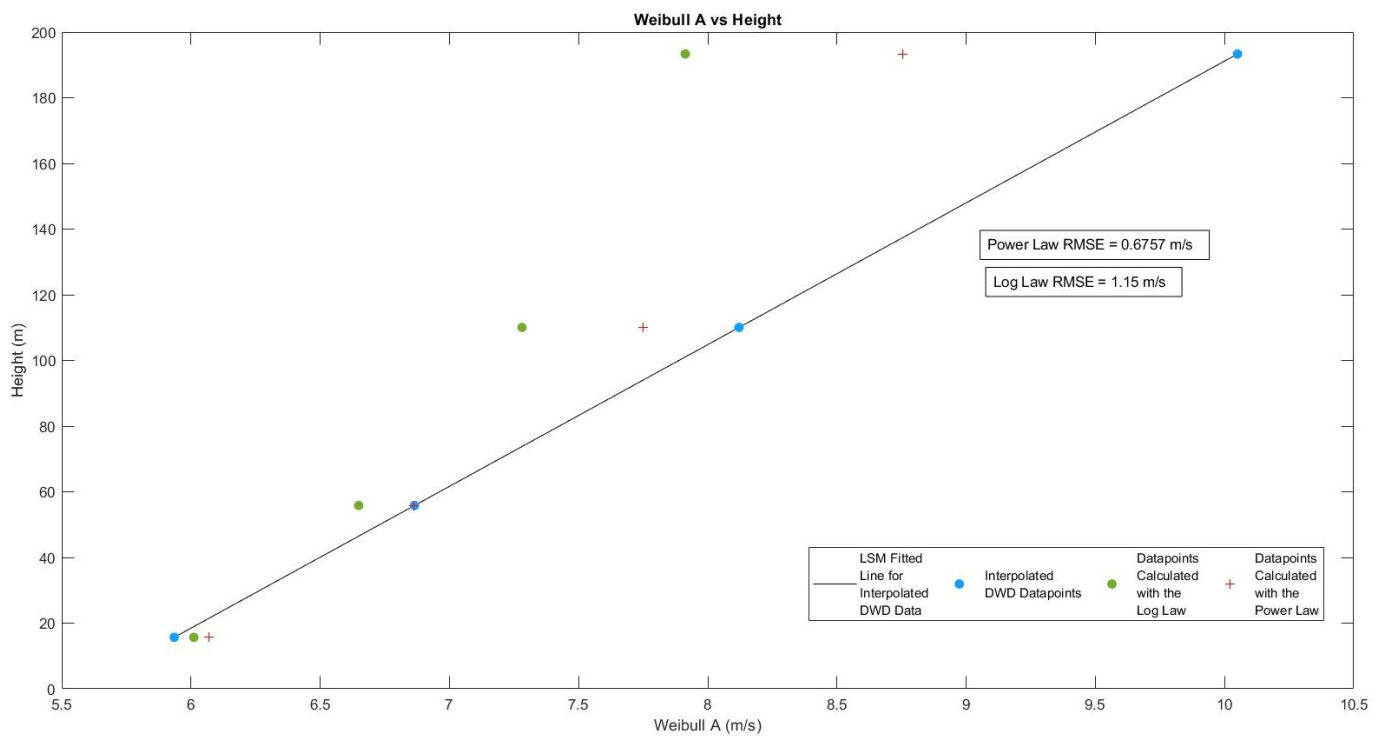


Figure A.4: Weibull A vs Height for Both Laws Rostock

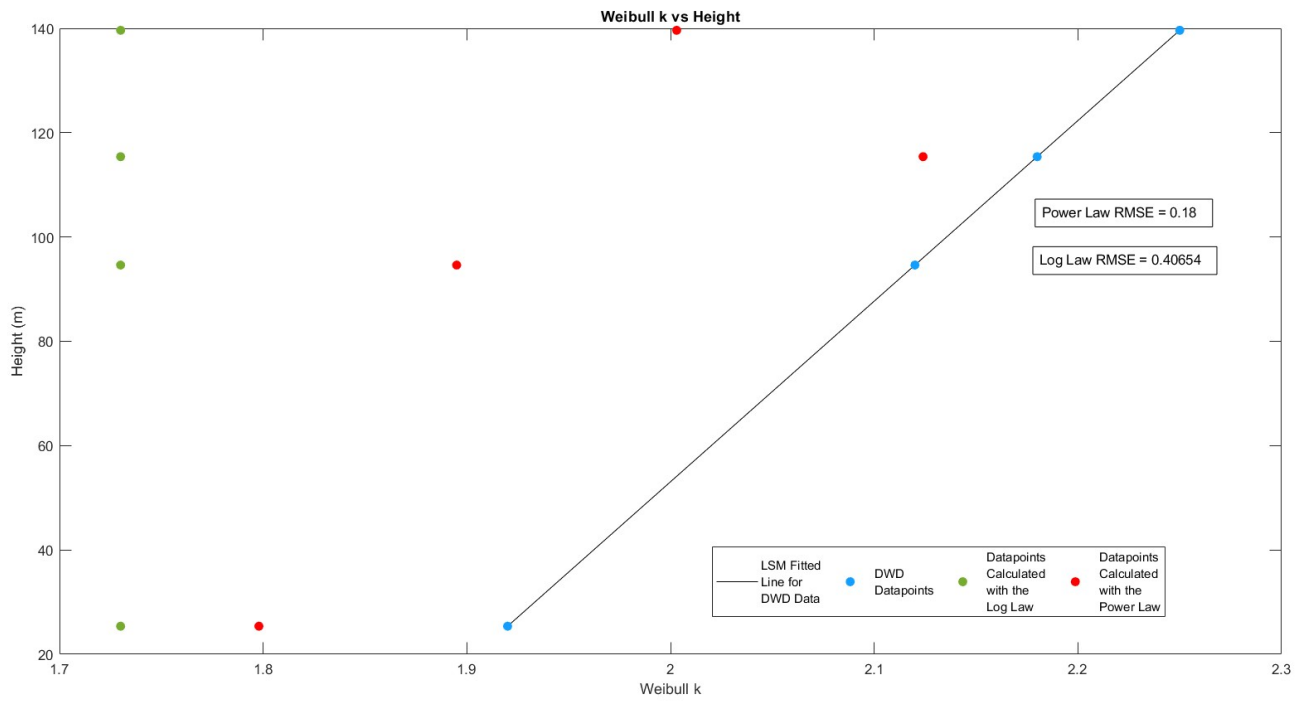


Figure A.5: Weibull k vs Height for Both Laws Rostock

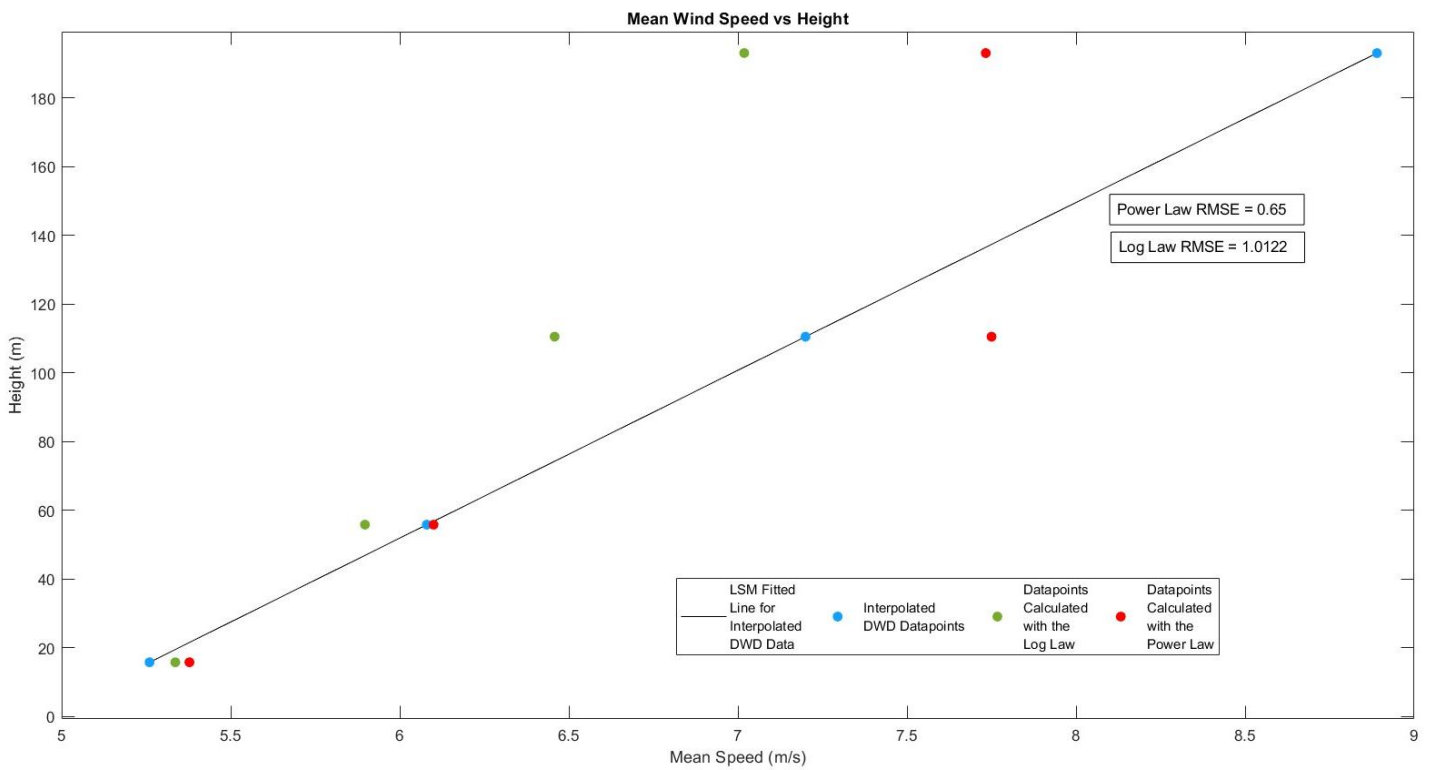


Figure A.6: Mean Speed vs Height for both Laws Rostock

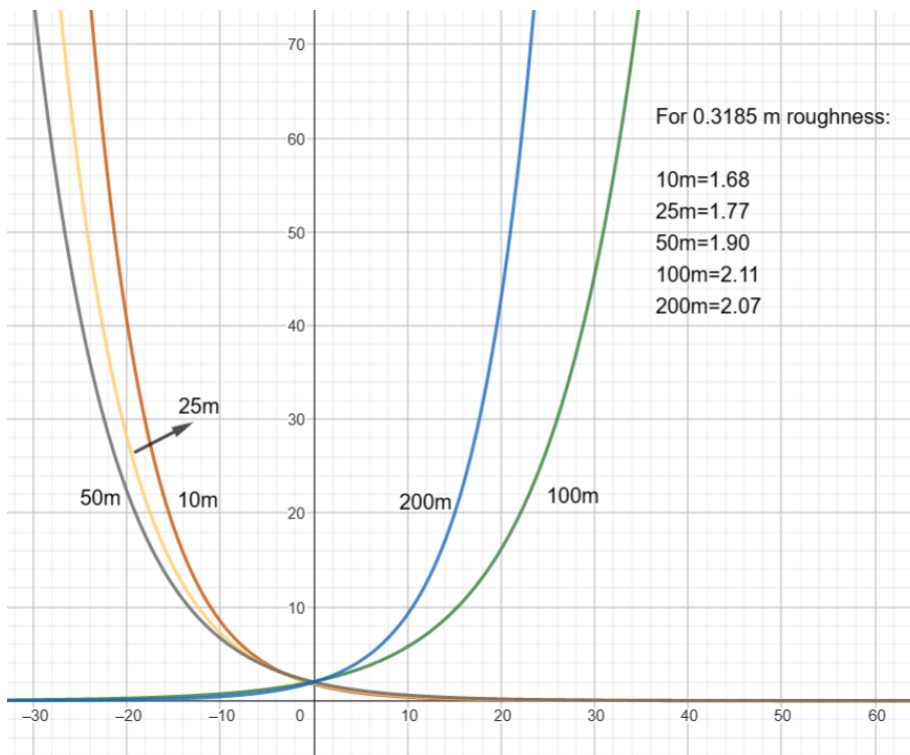


Figure A.7: Exponential curves fit for the Weibull k parameter of Schmücke

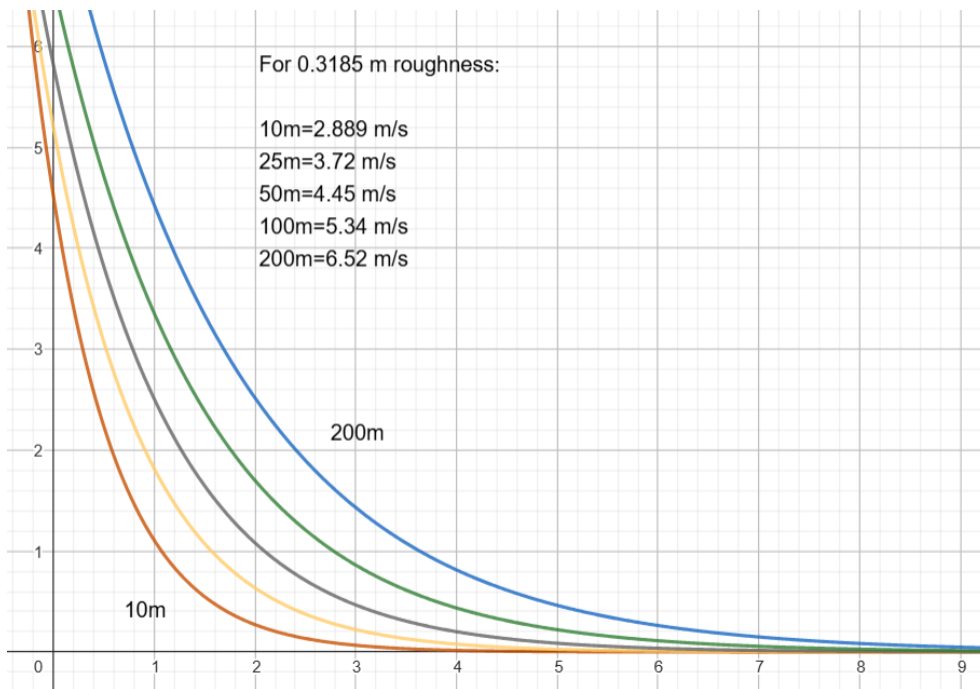


Figure A.8: Exponential curves fit for the mean wind speed parameter of Schmücke

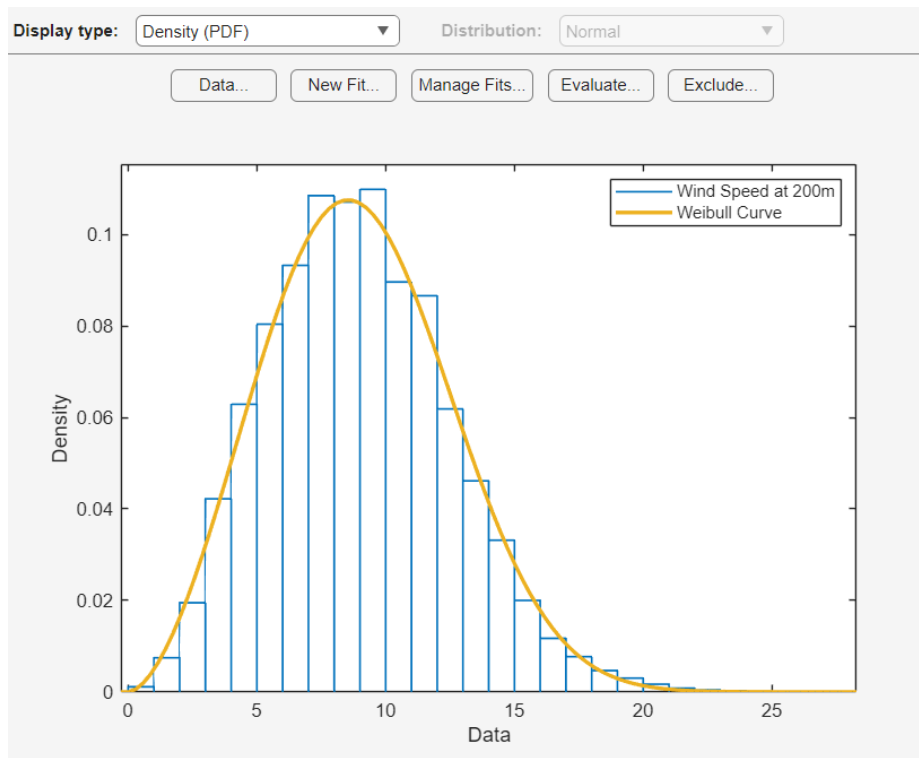


Figure A.9: The Weibull curve of Bremen wind data obtained from MATLAB Distribution Fitter

	Weibull A (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	10.05	7.56	7.36	7.08
Error at 200m		2.49	2.69	2.97
	Weibull k			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	2.12	2.041	2.166	1.84
Error at 200m		0.079	0.046	0.28
	Mean Speed (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	8.9	6.69	6.51	6.27
Error at 200m		2.21	2.39	2.63

Figure A.10: Parameters and errors of the combined laws for Rostock

	Weibull A (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	7.35	8.119	9.289	8.16
Error at 200m		0.769	1.939	0.81
	Weibull k			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	2.07	1.89	2.36	1.85
Error at 200m		0.18	0.29	0.22
	Mean Speed (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	6.52	7.186	8.22	7.22
Error at 200m		0.666	1.7	0.7

Figure A.11: Parameters and errors of the combined laws for Schmücke

	Weibull A (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	8.896	9.58	10.34	9.63
Error at 200m		0.684	1.444	0.734
	Weibull k			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	2.73	2.60	3.11	2.53
Error at 200m		0.13	0.38	0.20
	Mean Speed (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	7.92	8.504	9.26	8.54
Error at 200m		0.584	1.34	0.62

Figure A.12: Parameters and errors of the combined laws for Kahler Asten

	Weibull A (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	8.63	8.88	9.56	7.25
Error at 200m		0.25	0.93	1.38
	Weibull k			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	2.21	2.06	2.7	1.93
Error at 200m		0.15	0.49	0.28
	Mean Speed (m/s)			
Height (m)	Reference	Both Laws	Power Law	Log Law
200	7.65	7.88	8.5	6.43
Error at 200m		0.23	0.85	1.22

Figure A.13: Parameters and errors of the combined laws for Hannover